

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών Και Μηχανικών Υπολογιστών
8ο Εξάμηνο

Όραση Υπολογιστών

1ο Εργαστηριακό Project:

Εντοπισμός Σημείων Ενδιαφέροντος και Εξαγωγή Χαρακτηριστικών σε Εικόνες

Μέλη Ομάδας:

Γεωργίου Παναγιώτης - ΑΜ: 03122073

Μπάρκας Γεώργιος - ΑΜ: 03122139

Αθήνα, Μάρτιος 2026

ΠΕΡΙΕΧΟΜΕΝΑ

Μέρος 1: Ανίχνευση Ακμών σε Γκρίζες Εικόνες	2
1.1 Δημιουργία Εικόνων Εισόδου	2
1.1.1. Φόρτωση Αρχικής Εικόνας	2
1.1.2. Προσθήκη Θορύβου	2
1.2 Υλοποίηση Αλγορίθμου Ανίχνευσης Ακμών	3
1.2.1 Δημιουργία Γραμμικών Φίλτρων	4
1.2.2 Προσέγγιση της Laplacian της Εξομαλυμένης Εικόνας	4
1.2.3 Προσέγγιση των Σημείων Μηδενισμού	6
1.2.4 Απόρριψη Zero-crossings σε Ομαλές Περιοχές	6
1.3 Αξιολόγηση των Αποτελεσμάτων Ανίχνευσης Ακμών	7
1.3.1 Υπολογισμός των Αληθινών Ακμών	7
1.3.2 Ποσοτική Αξιολόγηση	8
1.3.3 Πειραματισμός και Σχολιασμός	8
1.4 Εφαρμογή των Αλγορίθμων Ανίχνευσης Ακμών σε Πραγματικές Εικόνες	11
1.4.1 Προετοιμασία Δεδομένων και Ground Truth	11
1.4.2 Βελτιστοποίηση Οπτικών Αποτελεσμάτων - Ποιοτική Αξιολόγηση	12
Μέρος 2: Ανίχνευση Σημείων Ενδιαφέροντος	14
2.1. Ανίχνευση Ακμών	14
2.1.1. Υπολογισμός Τανυστή J	14
2.1.2. Υπολογισμός ιδιοτιμών του τανυστή J	15
2.1.3. Κριτήριο γωνιότητας R	16
2.2. Πολυκλιμακωτή Ανίχνευση Γωνιών	18
2.2.1. Εφαρμογή ανιχνευτή Harris για διαφορετικές κλίμακες	18
2.2.2. Επιλογή Χαρακτηριστικής Κλίμακας	19
2.3. Ανίχνευση Blobs	20
2.4. Πολυκλιμακωτή Ανίχνευση Blobs	22
Μέρος 3: Κατηγοριοποίηση Εικόνων	23
3.1. Κατηγοριοποίηση με τοπικούς περιγραφητές	23
3.2. Κατηγοριοποίηση με χαρακτηριστικά από συνελικτικό νευρωνικό δίκτυο	25
3.2.1. Φόρτωση Μοντέλου και Προεπεξεργασία Εικόνων	25
3.2.2. Εξαγωγή Χαρακτηριστικών	26
3.2.3. Δημιουργία Συνόλων Train, Test και Αξιολόγηση	26
3.2.4. Feature Maps	26
3.2.5 Επίδραση Παραμορφώσεων στην Απόδοση	28

Μέρος 1: Ανίχνευση Ακμών σε Γκριζες Εικόνες

Στο πρώτο μέρος του συγκεκριμένου εργαστηριακού project θα ασχοληθούμε με Edge Detection σε Γκριζες (Grayscale) φωτογραφίες. Συγκεκριμένα, θα δουλέψουμε σε φωτογραφίες με θόρυβο (Gaussian Noise) αλλά και χωρίς, και θα υλοποιήσουμε έναν αλγόριθμο για Edge Detection, βασισμένο στους μηδενισμούς (zero-crossings) της Laplacian της εξομαλυμένης εικόνας.

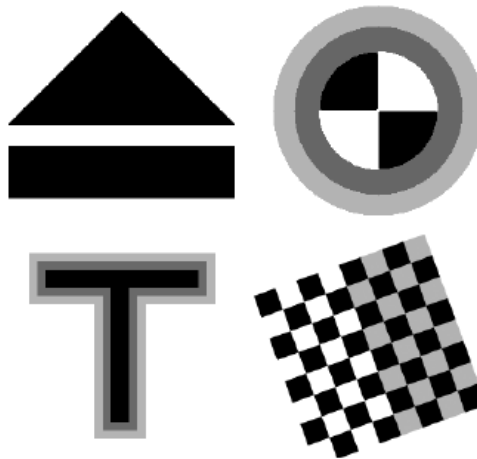
Σε ό,τι αφορά τη Λαπλασιανή, εξετάζονται δύο διαφορετικές προσεγγίσεις υπολογισμού της: η γραμμική μέσω συνέλιξης με τον πυρήνα Laplacian-of-Gaussian (LoG), και η μη-γραμμική μέσω μορφολογικών φίλτρων διαστολής και διάβρωσης. Όπως προαναφέρθηκε, αξιολογούμε την απόδοση των μεθόδων υπό συνθήκες θορύβου, χρησιμοποιώντας εικόνες με διαφορετικά επίπεδα PSNR, και συγκρίνουμε ποσοτικά τα αποτελέσματα με τις αληθινές ακμές, τόσο ποσοτικά μέσω του κριτηρίου Precision-Recall, αλλά και ποιοτικά με απλή εποπτεία των αποτελεσμάτων.

1.1 Δημιουργία Εικόνων Εισόδου

Αρχικά, διαβάσαμε τη δοθείσα εικόνα *edgetest_26.png* (I_0) και στη συνέχεια προσθέσαμε λευκό Gaussian θόρυβο με μηδενική μέση τιμή, ώστε να δημιουργήσουμε δύο θορυβώδεις εικόνες με PSNR 20 dB και 10 dB αντίστοιχα.

1.1.1. Φόρτωση Αρχικής Εικόνας

Η αρχική εικόνα I_0 φορτώθηκε σε μορφή grayscale και αποθηκεύτηκε ως πίνακας με τη βοήθεια της βιβλιοθήκης OpenCV. Η διάσταση της εικόνας είναι 512×512 pixels, με τιμές έντασης φωτεινότητας των pixels στο εύρος $[0, 255]$. Για την οπτικοποίηση της εικόνας, χρησιμοποιήσαμε τη βιβλιοθήκη Matplotlib και τη συνάρτηση *imshow*.



Αρχική Εικόνα (I_0) - *edgetest_26.png*

1.1.2. Προσθήκη Θορύβου

Στο συγκεκριμένο ερώτημα, μας ζητούσαν να προσθέσουμε θόρυβο στην αρχική εικόνα (I_0), ώστε να χρησιμοποιήσουμε τις θορυβώδεις εικόνες στον αλγόριθμο ανίχνευσης ακμών μας. Συγκεκριμένα, προσθέσαμε λευκό Gaussian θόρυβο $n(x, y)$ με μηδενική μέση τιμή και κατάλληλη τυπική απόκλιση σ_n . Η θορυβώδης εικόνα $I(x, y)$ προκύπτει από τη σχέση:

$$I(x, y) = I_0(x, y) + n(x, y)$$

Η ποσότητα του θορύβου καθορίζεται από τον λόγο Σήματος προς Θόρυβο (Peak-to-peak Signal to Noise Ratio - PSNR), ο οποίος ορίζεται ως:

$$PSNR = 20 \log_{10} \left(\frac{I_{max} - I_{min}}{\sigma_n} \right) \text{ (dB)} \quad (1)$$

όπου $I_{max} = \max_{x,y} I(x,y)$ και $I_{min} = \min_{x,y} I(x,y)$ οι μέγιστες και ελάχιστες τιμές φωτεινότητας της αρχικής εικόνας αντίστοιχα.

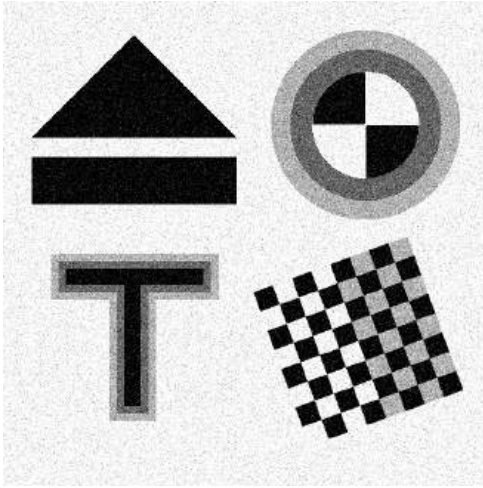
Στην εκφώνηση, μας ζητούν να εισάγουμε θόρυβο με συγκεκριμένο PSNR (για 20dB και 10dB αντίστοιχα), οπότε λύσαμε την παραπάνω εξίσωση ως προς την τυπική απόκλιση σ_n του θορύβου, καταλήγοντας στον εξής τύπο:

$$\sigma_n = \frac{I_{max} - I_{min}}{10^{\frac{PSNR}{20}}} \quad (2)$$

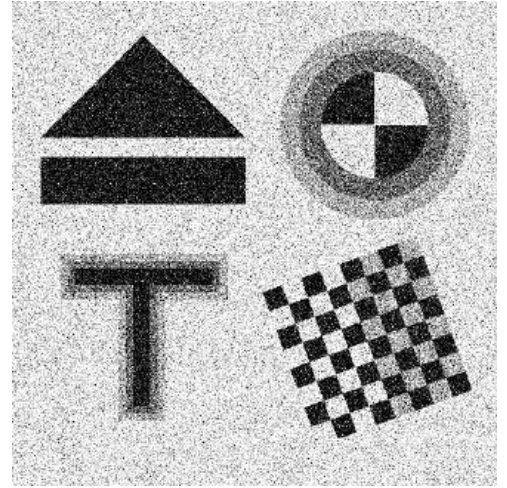
Αντικαθιστώντας στην παραπάνω σχέση τις τιμές του PSNR (20 dB και 10 dB) και τις μέγιστες/ελάχιστες τιμές της αρχικής εικόνας (εδώ που είναι grayscale, ήταν 255 και 0 αντίστοιχα), υπολογίσαμε τις αντίστοιχες τυπικές αποκλίσεις σ_n για τον θόρυβο που θα προσθέσουμε. Οι τιμές της σ_n που προέκυψαν (για εικόνα στο εύρος $[0, 255]$, δεν κανονικοποιήσαμε εδώ, θα μπορούσαμε, αλλά απλά θα γινόντουσαν οι τυπικές αποκλίσεις μικρότερες) ήταν:

- Για PSNR = 20 dB: $\sigma_n = 25.5$
- Για PSNR = 10 dB: $\sigma_n \approx 80.638$

Χρησιμοποιώντας τις παραπάνω τιμές (και τη συνάρτηση `numpy.random.normal`), δημιουργήσαμε τις 2 ζητούμενες εικόνες με **PSNR = 20 dB** και **PSNR = 10 dB**. Οι εικόνες αυτές παρουσιάζονται παρακάτω:



(a) Εικόνα με PSNR = 20 dB



(b) Εικόνα με PSNR = 10 dB

Όπως είναι αναμενόμενο, η ελάττωση του PSNR στα 10 dB αντιστοιχεί σε σημαντική αύξηση της διασποράς του προστιθέμενου θορύβου, "ρίχνοντας" οπτικά την ποιότητα της εικόνας, αλλά το σημαντικότερο που μας απασχολεί είναι το γεγονός ότι γίνονται οι ακμές της πιο δυσδιάκριτες.

1.2 Υλοποίηση Αλγορίθμου Ανίχνευσης Ακμών

Για την ανίχνευση ακμών, υλοποιήσαμε τη συνάρτηση `EdgeDetect`, η οποία προσεγγίζει τη Λαπλασιανή της εξομαλυνμένης εικόνας με δύο τρόπους: τη Γραμμική προσέγγιση (L_1) μέσω συνέλιξης με Laplacian-of-Gaussian (LoG), και τη Μη-γραμμική (L_2) μέσω μορφολογικών φίλτρων (διαστολή και διάβρωση). Στη συνέχεια, υπολογίσαμε τα σημεία μηδενισμού (zerocrossings) και απορρίψαμε όσα βρίσκονται σε σχετικά ομαλές περιοχές, εφαρμόζοντας το κατώφλι θ_{edge} για το μέτρο της κλίσης.

Η συνάρτηση, τελικά, θα δέχεται σαν ορίσματα την εικόνα εισόδου (I), την τυπική απόκλιση (σ) για το φίλτρο εξομάλυνσης, το κατώφλι (θ_{edge}) για την απόρριψη των μηδενισμών σε ομαλές περιοχές, και τον τύπο προσέγγισης (γραμμική ή μη-γραμμική), ενώ θα επιστρέφει έναν δυαδικό χάρτη ακμών (edge map) με τιμές 1 στα σημεία ακμών και 0 αλλιώς.

1.2.1 Δημιουργία Γραμμικών Φίλτρων (Gaussian και LoG)

Το πρώτο βήμα για την υλοποίηση του αλγορίθμου ανίχνευσης ακμών ήταν η δημιουργία των χροστικών αποκρίσεων δύο διακριτών γραμμικών φίλτρων: της δισδιάστατης Gaussian $G_\sigma(x, y)$ και της Laplacian-of-Gaussian (LoG) $h(x, y) = \nabla^2 G_\sigma(x, y)$.

Όπως έχουμε δει και στη θεωρία, η εφαρμογή διαφορικών τελεστών (όπως η Λαπλασιανή) ενισχύει τον υψίσυχνο θόρυβο της εικόνας. Για τον λόγο αυτό, καθίσταται αναγκαία η χρήση Γκαουσιανής κανονικοποίησης (Gaussian smoothing) πριν την παραγωγή. Λόγω της προσεταιριστικής ιδιότητας της συνέλιξης, η εφαρμογή της Λαπλασιανής σε μια εξομαλυμένη εικόνα είναι ισοδύναμη με τη συνέλιξη της εικόνας απευθείας με τη Λαπλασιανή της Γκαουσιανής (LoG).

Και στις δύο περιπτώσεις, η χωρική κλίμακα των φίλτρων καθορίζεται από την τυπική απόκλιση σ (που δέχεται η συνάρτησή μας σαν είσοδο). Το μέγεθος των kernel ($n \times n$ pixels) υπολογίστηκε μέσω της παρακάτω σχέσης:

$$n = \lceil 3 \cdot \sigma \rceil \cdot 2 + 1$$

Ο υπολογισμός αυτός διασφαλίζει πως τα όρια του "window" περιλαμβάνουν σχεδόν το σύνολο της ενέργειας της Γκαουσιανής κατανομής (περίπου 99.7% βάσει του κανόνα των 3σ), ελαχιστοποιώντας το σφάλμα αποκοπής (truncation error).

Στο ερώτημα αυτό λοιπόν, υλοποιήσαμε τη συνάρτηση `gaussian_kernel_generator()`, που μας επιστρέφει τους 2 ζητούμενους πυρήνες. Ενώ, παίρνει μονάχα σαν είσοδο, την τιμή της τυπικής απόκλισης σ .

Υλοποίηση Φίλτρων:

- **Δισδιάστατη Gaussian (G_σ):** Η συνεχής συνάρτηση της δισδιάστατης Γκαουσιανής ορίζεται ως:

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Εχμεταλλευόμενοι την ιδιότητα διαχωρισιμότητας (separability) της Γκαουσιανής συνάρτησης, δημιουργήσαμε πρώτα έναν 1D πυρήνα και στη συνέχεια υπολογίσαμε το εξωτερικό γινόμενο με τον ανάστροφό του, παράγοντας τον τελικό $n \times n$, 2D πυρήνα.

- **Laplacian-of-Gaussian (LoG):** Η χροστική απόκριση του φίλτρου LoG προκύπτει από την εφαρμογή του τελεστή Laplace ($\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$) στην Γκαουσιανή συνάρτηση, καταλήγοντας στον αναλυτικό τύπο:

$$h(x, y) = \nabla^2 G_\sigma(x, y) = \frac{x^2 + y^2 - 2\sigma^2}{2\pi\sigma^6} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (3)$$

Για τον υπολογισμό του πυρήνα LoG, προτιμήσαμε την άμεση αναλυτική υλοποίηση της συνάρτησης σε ένα δισδιάστατο πλέγμα συντεταγμένων (meshgrid) με κέντρο το $(0, 0)$.

Ενώ, αφού είχαμε τις συντεταγμένες x και y , υλοποιήθηκε ο παραπάνω αναλυτικός μαθηματικός τύπος (3).

Η επιλογή της άμεσης υλοποίησης έναντι της εφαρμογής ενός Λαπλασιανού τελεστή πάνω στον Γκαουσιανό πυρήνα (μέσω της `cv2.Laplacian`) προτιμήθηκε (όπως μας παρότρυνε και η εκφώνηση), καθώς αποτρέπει την εισαγωγή πρόσθετων σφαλμάτων προσέγγισης.

Η συνάρτησή μας λοιπόν, στο συγκεκριμένο ερώτημα, επιστρέφει τους δύο ζητούμενους πυρήνες, με τη διαδικασία όπως ακριβώς περιγράφηκε παραπάνω.

1.2.2 Προσέγγιση της Laplacian της Εξομαλυμένης Εικόνας

Στη συνέχεια, στόχος του αλγορίθμου είναι ο υπολογισμός της Λαπλασιανής της εξομαλυμένης εικόνας $I_\sigma(x, y) = G_\sigma * I(x, y)$, προκειμένου να εντοπιστούν οι ακμές μέσω των σημείων μηδενισμού (zero-crossings) της δεύτερης παραγωγού. Για τον σκοπό αυτό, υλοποιήσαμε και συγκρίναμε δύο διαφορετικές προσεγγίσεις: μία Γραμμική και μία Μη-Γραμμική (μορφολογική).

Για το συγκεκριμένο ζητούμενο, υλοποιήσαμε τη συνάρτηση `apply_filters()`, η οποία προσεγγίζει τη Λαπλασιανή της εξομαλυμένης εικόνας με δύο τρόπους: τη Γραμμική προσέγγιση (L_1) μέσω συνέλιξης με Laplacian-of-Gaussian (LoG), και τη Μη-γραμμική (L_2) μέσω μορφολογικών φίλτρων (διαστολή και διάβρωση).

Η συγκεκριμένη συνάρτηση παίρνει σαν ορίσματα την εικόνα εισόδου (I), την τυπική απόκλιση (σ) για το φίλτρο εξομάλυνσης και καλεί την συνάρτηση `gaussian_kernel_generator()` για να πάρει τους δύο πυρήνες. Στη συνέχεια, υπολογίζει τις δύο προσεγγίσεις της Λαπλασιανής και επιστρέφει τα αποτελέσματα.

i) Γραμμική Προσέγγιση (L_1):

Βάσει των γραμμικών συστημάτων, που έχουμε δει και στο μάθημα, η παραγωγή της εξομαλυσμένης εικόνας είναι μαθηματικά ισοδύναμη με τη συνέλιξη της αρχικής θορυβώδους εικόνας I απευθείας με τον πυρήνα Laplacian-of-Gaussian (LoG) h . Συνεπώς, η προσέγγιση L_1 υπολογίζεται ως εξής:

$$L_1 = \nabla^2(G_\sigma * I) = (\nabla^2 G_\sigma) * I = h * I$$

Προγραμματιστικά, αυτό επιτεύχθηκε εφαρμόζοντας τη συνάρτηση `cv2.filter2D` με τον πυρήνα LoG που δημιουργήσαμε στο προηγούμενο βήμα.

ii) Μη-Γραμμική Προσέγγιση (L_2):

Εναλλακτικά, η Λαπλασιανή μπορεί να εκτιμηθεί με χρήση μη-γραμμικών μορφολογικών φίλτρων πάνω στην εξομαλυσμένη (με την 2D Gaussian) εικόνα I_σ . Χρησιμοποιώντας ένα structuring element B μεγέθους 3×3 , η Λαπλασιανή L_2 προσεγγίζεται από τη διαφορά της μορφολογικής διαστολής (dilation, $\oplus$) και της μορφολογικής διάβρωσης (erosion, $\ominus$):

$$L_2 = (I_\sigma \oplus B) + (I_\sigma \ominus B) - 2I_\sigma$$

Αυτή η τεχνική (γνωστή και ως μορφολογική Λαπλασιανή) αξιολογεί τις τοπικές διακυμάνσεις της φωτεινότητας εντός της γειτονιάς του δομικού στοιχείου.

Στην παραπάνω εξίσωση, το $I_\sigma(x, y) = G_\sigma * I(x, y)$ αποτελεί την εξομαλυσμένη εικόνα, ενώ το B είναι της μορφής:

$$B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Η συγκεκριμένη επιλογή του B εξασφαλίζει ότι κατά τις μορφολογικές πράξεις λαμβάνονται υπόψη αποκλειστικά τα άμεσα γειτονικά pixels. Λόγω αυτού, σε ομαλές περιοχές όπου η φωτεινότητα παραμένει πρακτικά σταθερή, η μορφολογική Λαπλασιανή τείνει στο μηδέν ($L_2 \approx 0$).

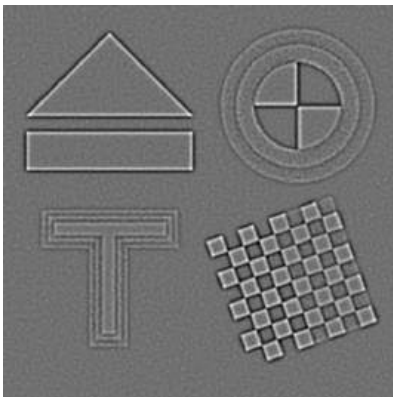
Αντίθετα, στις περιοχές όπου υπάρχουν απότομες μεταβολές της φωτεινότητας (δηλαδή στις ακμές), δημιουργούνται μεγάλες διαφορές μεταξύ των γειτονικών pixels, με αποτέλεσμα η μορφολογική Λαπλασιανή να παίρνει σημαντικές θετικές ή αρνητικές τιμές, ανάλογα με τη φορά της μεταβολής.

Λίγο πιο γεωμετρικά, η χρήση του πίνακα B βοηθάει ώστε να γίνει μεγένθυση και σμίκρυνση των φωτεινών περιοχών κάθετα και οριζόντια, χωρίς να επηρεάζονται οι διαγώνιες κατευθύνσεις. Η αφαίρεση στο τέλος χρησιμοποιείται για να παραμείνουν ανεπηρέαστες οι περιοχές που δεν περιλαμβάνουν ακμές.

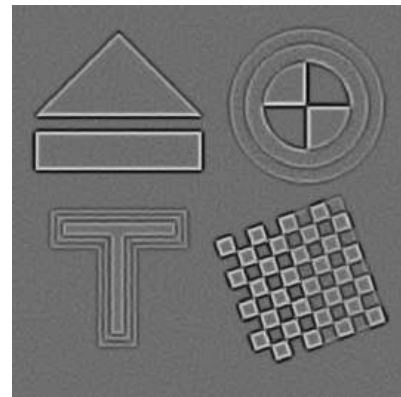
Λεπτομέρεια Υλοποίησης:

Κατά την υλοποίηση της Μη-Γραμμικής προσέγγισης, μετατρέψαμε την εικόνα I_σ σε float64 ή `cv2.CV_64F` πριν την εφαρμογή των αλγεβρικών πράξεων. Αν η εικόνα παρέμεινε στον προεπιλεγμένο τύπο ακέραιων 8-bit (`uint8`), οι αφαιρέσεις θα προκαλούσαν σφάλματα υπερχείλισης (overflow/underflow), οδηγώντας σε λανθασμένα αποτελέσματα κατά τον υπολογισμό των σημείων μηδενισμού.

Οπτικοποίηση των δύο προσεγγίσεων της Λαπλασιανής για τη θορυβώδη εικόνα (PSNR = 20 dB) και τυπική απόκλιση $\sigma = 2.0$ παρουσιάζεται παρακάτω (με κατάλληλες κλίμακες, γιατί είχαμε και αρνητικές τιμές, κανονικοποιήσαμε στο $[0, 255]$ για το visualization).



(a) Γραμμική προσέγγιση (LoG)



(b) Μη-γραμμική προσέγγιση

1.2.3 Προσέγγιση των Σημείων Μηδενισμού (Zero-crossings)

Έχοντας υπολογίσει την προσέγγιση της Λαπλασιανής L (είτε μέσω της Γραμμικής είτε μέσω της Μη-Γραμμικής μεθόδου), το επόμενο βήμα του αλγορίθμου συνίσταται στον εντοπισμό των σημείων μηδενισμού (zero-crossings). Στα σημεία αυτά, η δεύτερη παράγωγος της φωτεινότητας αλλάζει πρόσημο, υποδηλώνοντας την ύπαρξη ενός τοπικού μεγίστου στην πρώτη παράγωγο, το οποίο αντιστοιχεί θεωρητικά σε μία ακμή.

Η συνάρτηση `zero_crossing_edge_map()` υλοποιεί τη διαδικασία εντοπισμού των zero-crossings, λαμβάνοντας ως είσοδο την προσέγγιση της Λαπλασιανής L και επιστρέφοντας έναν δυαδικό χάρτη ακμών.

Για την υπολογιστική προσέγγιση των zero-crossings, ακολουθήθηκε η εξής μορφολογική διαδικασία:

1. **Δημιουργία Δυαδικής Εικόνας Προσέμου (X):** Αρχικά, κατασκευάστηκε μία δυαδική μάσκα X , η οποία διαχωρίζει την εικόνα με βάση το πρόσημο της Λαπλασιανής. Συγκεκριμένα, λαμβάνει την τιμή 1 (True) στα pixels όπου η L είναι μη αρνητική και 0 (False) διαφορετικά:

$$X = (L \geq 0)$$

2. **Εύρεση Περιγράμματος (Y):** Στη συνέχεια, τα σημεία αλλαγής προσέμου εντοπίστηκαν εξάγοντας το περίγραμμα ∂X της δυαδικής εικόνας X . Το περίγραμμα αυτό (το οποίο είναι συμμετρικό ως προς το X και το συμπλήρωμά του X^c) υπολογίστηκε μορφολογικά, αφαιρώντας τη διάβρωση της X από τη διαστολή της, χρησιμοποιώντας εκ νέου το δομικό στοιχείο B :

$$Y = (X \oplus B) - (X \ominus B) \approx \partial X$$

Η προκύπτουσα δυαδική εικόνα Y περιέχει την τιμή 1 αποκλειστικά στα pixels όπου καταγράφεται μετάβαση του προσέμου της Λαπλασιανής (από θετικό σε αρνητικό ή αντίστροφα). Τα σημεία αυτά ταυτοποιούνται ως τα zero-crossings της εξομαλυμένης εικόνας και αποτελούν το σύνολο των υποψήφιων ακμών.

1.2.4 Απόρριψη Zero-crossings σε Ομαλές Περιοχές

Η δυαδική εικόνα Y που προέκυψε από το προηγούμενο βήμα περιλαμβάνει όλα τα σημεία μηδενισμού της Λαπλασιανής. Ωστόσο, στην πράξη, zero-crossings ενδέχεται να εμφανιστούν και σε περιοχές της εικόνας με πολύ μικρές, ανεπαίσθητες διακυμάνσεις της φωτεινότητας (π.χ. λόγω υπολειπόμενου θορύβου), οι οποίες δεν αντιστοιχούν σε πραγματικές ακμές.

Για την απόρριψη αυτών των ψευδών ακμών, επιβάλουμε, όπως μας ζητείται, ένα επιπλέον κριτήριο, βασισμένο στο μέτρο της κλίσης της εξομαλυμένης εικόνας I_σ . Μία πραγματική ακμή χαρακτηρίζεται από υψηλό ρυθμό μεταβολής της φωτεινότητας. Επομένως, ως τελικές ακμές επιλέχθηκαν αποκλειστικά τα pixels (i, j) για τα οποία ικανοποιούνται ταυτόχρονα οι εξής δύο συνθήκες:

$$Y[i, j] = 1 \quad \text{και} \quad \|\nabla I_\sigma[i, j]\| > \theta_{edge} \cdot \max_{x, y} \|\nabla I_\sigma\|$$

όπου:

- $\|\nabla I_\sigma[i, j]\|$ είναι το μέτρο του διανύσματος της κλίσης στο σημείο (i, j) , το οποίο υπολογίστηκε προγραμματιστικά (μέσω της `numpy.gradient`) ως $\sqrt{(\frac{\partial I_\sigma}{\partial x})^2 + (\frac{\partial I_\sigma}{\partial y})^2}$.
- $\theta_{edge} \in [0, 1]$ είναι μία παράμετρος κατωφλίου (threshold) που καθορίζει το ελάχιστο αποδεκτό ποσοστό της μέγιστης κλίσης της εικόνας.

Λεπτομέρεια Υλοποίησης:

Σε επίπεδο προγραμματιστικού κώδικα, η παραπάνω συνθήκη υλοποιήθηκε βελτιστοποιημένα (pixel, pixel) κάνοντας χρήση της συνάρτησης `np.where` της βιβλιοθήκης NumPy. Δεδομένου ότι ο πίνακας `edge_map` (που αντιστοιχεί στο Y) είναι ήδη δυαδικός (binary), χρησιμοποιήσαμε τον τελεστή bitwise AND (`&`) μεταξύ των δύο πινάκων συνθηκών.

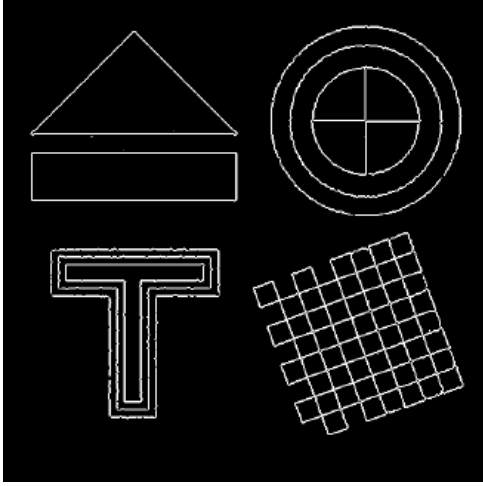
```
edge_map = np.where((edge_map == 1) & (grad_mag > threshold), 1, 0).astype(np.uint8)
```

Η εφαρμογή της παραπάνω κατωφλίωσης καταλήγει στην τελική δυαδική εικόνα ακμών D , η οποία αποτελεί και την έξοδο της συνάρτησης `EdgeDetect`. Αυξάνοντας την παράμετρο θ_{edge} , ο αλγόριθμος γίνεται πιο αυστηρός, διατηρώντας μόνο τις πιο έντονες ακμές της εικόνας και καταστέλλοντας περαιτέρω τον θόρυβο.

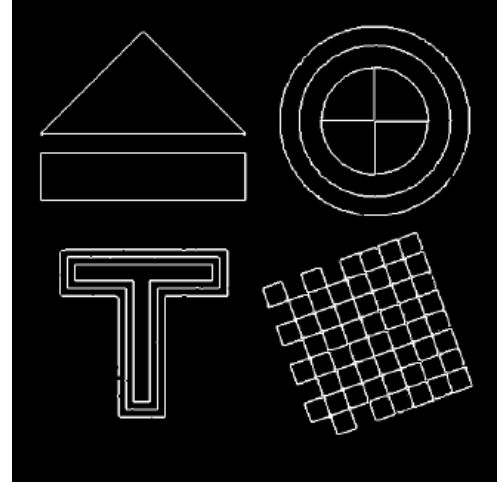
Συνολική Υλοποίηση της Συνάρτησης EdgeDetect

Συνδυάζοντας όλα τα παραπάνω επιμέρους βήματα (ενότητες 1.2.1 έως 1.2.4), κατασκευάστηκε η ενιαία συνάρτηση EdgeDetect. Η συνάρτηση αυτή υλοποιεί την πλήρη ροή (pipeline) του αλγορίθμου: δέχεται ως ορίσματα την εικόνα εισόδου I , την τυπική απόκλιση σ του Γκαουσιανού φίλτρου, την παράμετρο κατωφλίου θ_{edge} , καθώς και τον επιθυμητό τύπο προσέγγισης της Λαπλασιανής ('linear' ή 'non-linear'). Ως έξοδο, επιστρέφει την τελική δυαδική εικόνα ακμών D , με τιμή 1 αποκλειστικά στα σημεία που έχουν ταυτοποιηθεί ως έγκυρες ακμές.

Για την επιβεβαίωση της ορθής λειτουργίας του αλγορίθμου (γιατί παραπάνω διερεύνηση θα πραγματοποιηθεί και παρακάτω), εφαρμόσαμε τη συνάρτηση EdgeDetect στη θορυβώδη εικόνα με PSNR = 20 dB, χρησιμοποιώντας ενδεικτικές τιμές για τις παραμέτρους (π.χ. $\sigma = 1.5$, $\theta_{edge} = 0.2$). Οι τελικοί δυαδικοί χάρτες ακμών που παρήχθησαν και για τις δύο μεθόδους προσέγγισης παρουσιάζονται στο παρακάτω σχήμα:



(a) Τελικές Ακμές - Γραμμική Προσέγγιση



(b) Τελικές Ακμές - Μη-Γραμμική Προσέγγιση

1.3 Αξιολόγηση των Αποτελεσμάτων Ανίχνευσης Ακμών

Σε αυτό το μέρος της εργασίας, προχωρήσαμε στην ποσοτική αξιολόγηση της απόδοσης του αλγορίθμου ανίχνευσης ακμών που υλοποιήσαμε στο προηγούμενο βήμα. Για τον σκοπό αυτό, ήταν απαραίτητο να δημιουργήσουμε μια εικόνα αναφοράς που να περιέχει τις πραγματικές ακμές (Ground Truth) της αρχικής καθαρής εικόνας I_0 . Στη συνέχεια, συγκρίναμε τις ανιχνευθείσες ακμές με τις αληθινές ακμές, υπολογίζοντας τις μετρικές *Precision* και *Recall*, καθώς και ένα συνολικό κριτήριο ποιότητας (C') για διαφορετικές τιμές των παραμέτρων σ και θ_{edge} .

1.3.1 Υπολογισμός των Αληθινών Ακμών (Ground Truth)

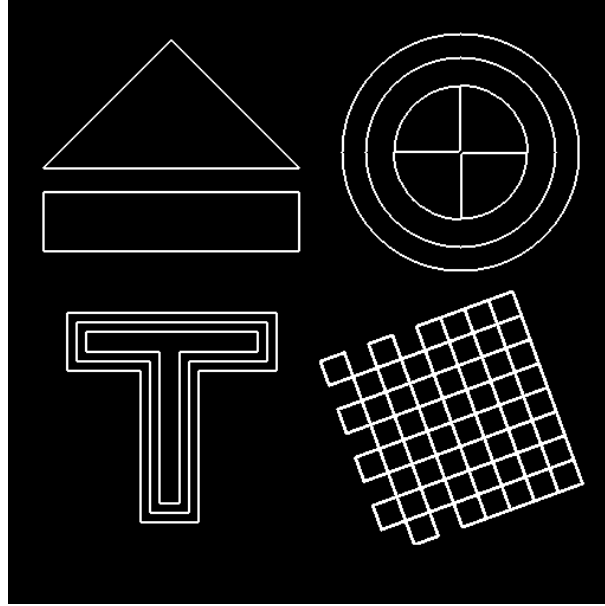
Για τον υπολογισμό της δυαδικής εικόνας αληθινών ακμών T , χρησιμοποιήσαμε την αρχική καθαρή (χωρίς θόρυβο) εικόνα I_0 .

Αρχικά, εφαρμόσαμε έναν απλό edge operator, υπολογίζοντας τη διαφορά μεταξύ της μορφολογικής διαστολής και της μορφολογικής διάβρωσης της I_0 με το ίδιο δομικό στοιχείο B , που είχαμε και νωρίτερα:

$$M = (I_0 \oplus B) - (I_0 \ominus B)$$

Στη συνέχεια, η τελική δυαδική εικόνα T προέκυψε μέσω thresholding της M με μια κατάλληλη παράμετρο $\theta_{realedge}$ (στον κώδικά μας χρησιμοποιήθηκε ενδεικτικά η τιμή $\theta_{realedge} = 2.0$, που κρίθηκε οπτικά η κατάλληλη που κρατάει μόνο τις πραγματικές ακμές):

$$T = (M > \theta_{realedge})$$



Εικόνα Αληθινών Ακμών (T) - Ground Truth

1.3.2 Ποσοτική Αξιολόγηση (Precision & Recall)

Έχοντας πλέον στη διάθεσή μας τις αληθινές ακμές T και τις ακμές D που ανίχνευσε ο αλγόριθμός μας στις θορυβώδεις εικόνες, προχωρήσαμε σε ποσοτική αξιολόγηση υπολογίζοντας τις μετρικές *Precision* και *Recall*.

Θεωρώντας τα D και T ως διακριτά σύνολα, όπου τα στοιχεία τους είναι τα pixels με τιμή 1, οι μετρικές υπολογίστηκαν μέσω του cardinality των συνόλων, ο οποίος προγραμματιστικά υλοποιήθηκε αθροίζοντας τα λογικά '1' του δυαδικού πίνακα (`array.sum()`).

- **Precision ($Pr(D|T)$):** Εκφράζει το ποσοστό των ανιχνευθεισών ακμών D που είναι όντως πραγματικές.

$$Pr(D|T) = \frac{\text{card}(D \cap T)}{\text{card}(D)}$$

- **Recall ($Pr(T|D)$):** Εκφράζει το ποσοστό των πραγματικών ακμών T που καταφέραμε να ανιχνεύσουμε.

$$Pr(T|D) = \frac{\text{card}(D \cap T)}{\text{card}(T)}$$

Το τελικό συνολικό κριτήριο ποιότητας του αποτελέσματος (C) ορίστηκε ως ο μέσος όρος των δύο παραπάνω ποσοστών:

$$C = \frac{Pr(D|T) + Pr(T|D)}{2}$$

Από πλευρά κώδικα, υλοποιήσαμε τη συνάρτηση `calculate_precision_recall()`, η οποία δέχεται ως είσοδο τις δυαδικές εικόνες D και T , υπολογίζει τις μετρικές Precision και Recall, και επιστρέφει αυτές καθώς και το συνολικό κριτήριο C .

1.3.3 Πειραματισμός και Σχολιασμός (PSNR 20 dB & 10 dB)

Στο τελικό στάδιο του πρώτου μέρους, πειραματιστήκαμε εκτενώς με τις τιμές των παραμέτρων σ και θ_{edge} για τα δύο διαφορετικά επίπεδα θορύβου (PSNR = 20 dB και PSNR = 10 dB), με στόχο τη μεγιστοποίηση του κριτηρίου C . Ελέγχοντας βέβαια παράλληλα και την οπτική ποιότητα των ανιχνευθεισών ακμών, προκειμένου να καταλήξουμε σε μια τελική εκτίμηση της απόδοσης του αλγορίθμου μας υπό διαφορετικές συνθήκες θορύβου.

Για τον εντοπισμό των βέλτιστων παραμέτρων λειτουργίας του αλγορίθμου, εφαρμόστηκε αναζήτηση τύπου grid search σε ένα καθορισμένο εύρος τιμών $[\sigma_{min}, \sigma_{max}] \times [\theta_{min}, \theta_{max}]$. Για κάθε εξεταζόμενο ζεύγος (σ, θ) , υπολογίστηκε η μετρική αξιολόγησης C που αναλύσαμε παραπάνω. Ο βέλτιστος συνδυασμός επιλέχθηκε με μοναδικό κριτήριο τη μεγιστοποίηση της συνολικής απόδοσης:

$$(\sigma_{best}, \theta_{best}) = \arg \max_{\sigma, \theta} C$$

Για τις δοκιμές μας, επιλέξαμε ένα λογικό εύρος τιμών για το σ (συγκεκριμένα από 0.5 έως 4.0, παίρνοντας 20 τιμές) και για το θ_{edge} (συγκεκριμένα από 0.1 έως 0.5 παίρνοντας 20 τιμές), προκειμένου να καλύψουμε τόσο ήπιες όσο και πιο αυστηρές συνθήκες ανίχνευσης ακμών. Οι τιμές αυτές επιλέχθηκαν λαμβάνοντας υπόψη και τις ενδεικτικές τιμές που μας έδινε η άσκηση σε σημεία αναφοράς.

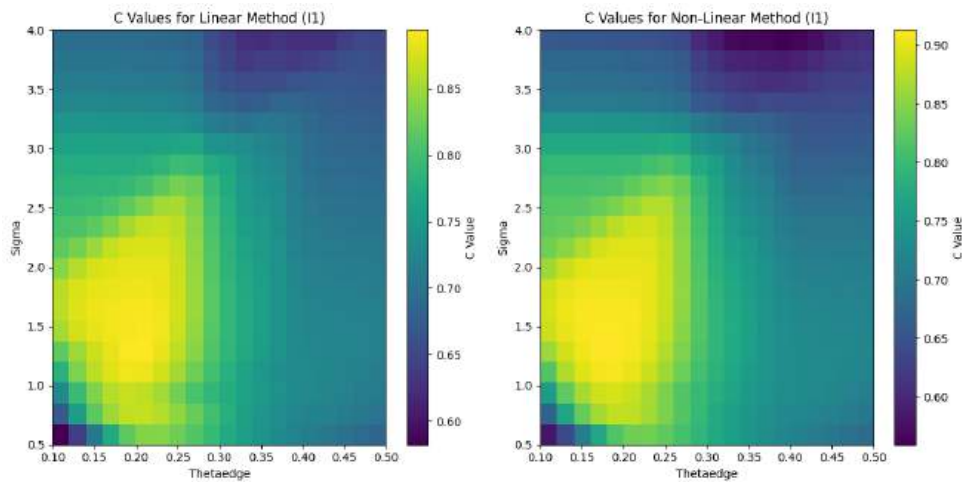
Οι ενδεικτικές τιμές των 0.2 για το θ_{edge} και 1.5 για το σ για την I_1 και 0.2 για το θ_{edge} και 3.0 για το σ για την I_2 που χρησιμοποιήσαμε παραπάνω, δοκιμάστηκαν ότι δίνουν πολύ καλά αποτελέσματα και επιβεβαιώθηκαν ότι είναι κοντά στις βέλτιστες τιμές, απλά η παραπάνω διαδικασία grid search μας επέτρεψε να εντοπίσουμε με μεγαλύτερη ακρίβεια τις ιδανικές παραμέτρους για κάθε επίπεδο θορύβου.

Για να αναπαραστήσουμε οπτικά τα αποτελέσματα του πειραματισμού, δημιουργήσαμε δύο heatmaps (ένα για κάθε επίπεδο θορύβου), όπου στον άξονα y απεικονίζονται οι τιμές του σ , στον άξονα x οι τιμές του θ_{edge} , και το χρώμα αντιστοιχεί στην τιμή της μετρικής C .

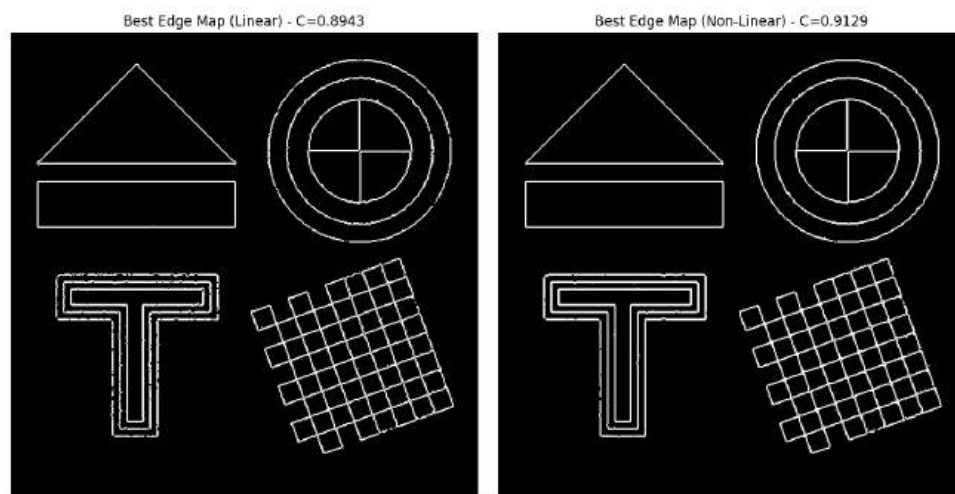
Αποτελέσματα για PSNR = 20 dB, δηλαδή για την εικόνα I_1 :

Για το σχετικά χαμηλό επίπεδο θορύβου (20 dB), οι βέλτιστες παράμετροι που εντοπίσαμε ήταν $\sigma = 1.24$ και $\theta_{edge} = 0.21$ για την υλοποίηση με τη γραμμική μέθοδο. Ενώ, για τη μη-γραμμική μέθοδο, οι βέλτιστες παράμετροι ήταν $\sigma = 1.24$ και $\theta_{edge} = 0.18$.

Σε αυτές τις συνθήκες, η μετρική C έφτασε το 0.894 για τη Γραμμική προσέγγιση και το 0.913 για τη Μη-Γραμμική προσέγγιση.



Αναπαράσταση Heatmap της μετρικής C (Grid Search) για την εικόνα I_1 (PSNR = 20 dB).

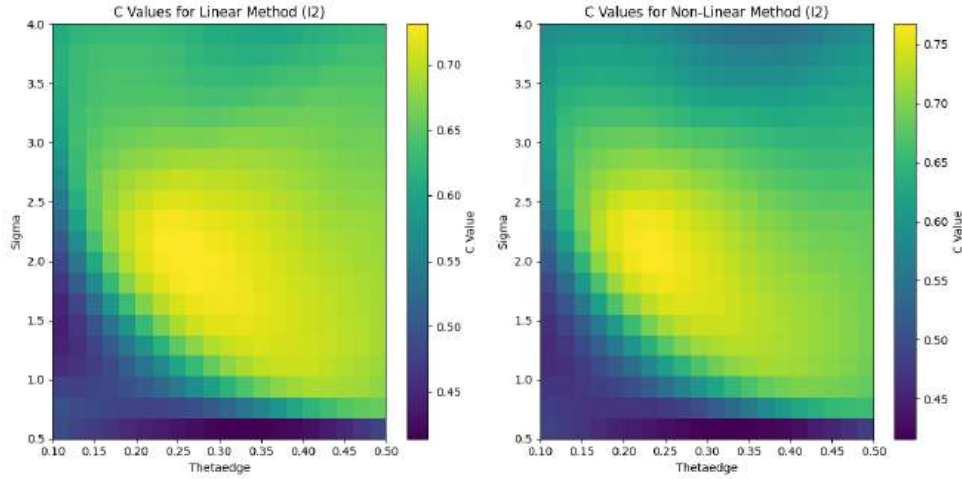


Τελικοί χάρτες ακμών (Edge Maps) για την I_1 με χρήση των βέλτιστων παραμέτρων (linear και non-linear αντίστοιχα).

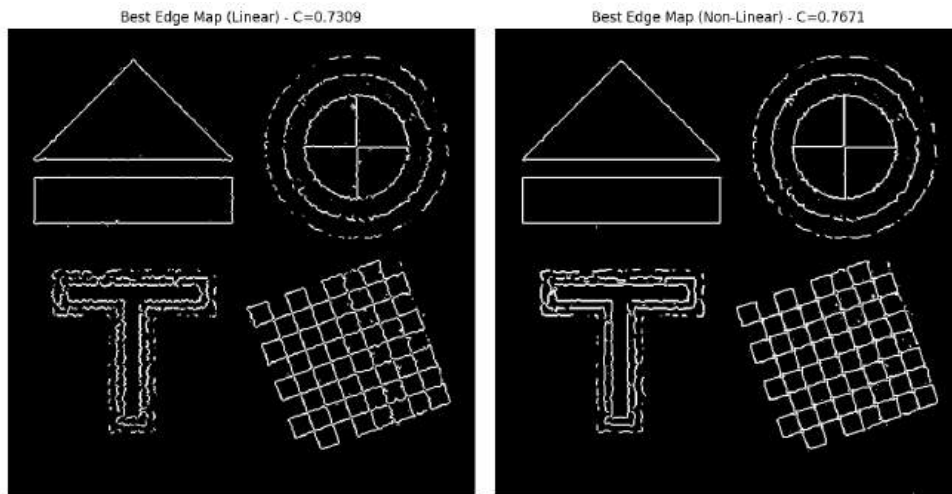
Αποτελέσματα για PSNR = 10 dB, δηλαδή για την εικόνα I_2 :

Καθώς ο θόρυβος αυξήθηκε δραματικά (PSNR = 10 dB), χρειάστηκε να αναπροσαρμόσουμε τις παραμέτρους. Οι βέλτιστες παράμετροι που εντοπίσαμε ήταν $\sigma = 1.97$ και $\theta_{edge} = 0.25$ για την υλοποίηση με τη γραμμική μέθοδο. Ενώ, για τη μη-γραμμική μέθοδο, οι βέλτιστες παράμετροι ήταν $\sigma = 2.16$ και $\theta_{edge} = 0.23$.

Σε αυτές τις συνθήκες, η μετρική C έφτασε το 0.731 για τη Γραμμική προσέγγιση και το 0.767 για τη Μη-Γραμμική προσέγγιση.



Αναπαράσταση Heatmap της μετρικής C (Grid Search) για την εικόνα I_2 (PSNR = 10 dB).



Τελικοί χάρτες ακμών (Edge Maps) για την I_2 με χρήση των βέλτιστων παραμέτρων (linear και non-linear αντίστοιχα).

Συνοπτικός Πίνακας Αποτελεσμάτων:

Για την καλύτερη εποπτεία των πειραμάτων, τα βέλτιστα ζεύγη παραμέτρων (σ, θ_{edge}) και η αντίστοιχη μέγιστη τιμή του κριτηρίου C για κάθε συνδυασμό εικόνας και μεθόδου συνοψίζονται στον παρακάτω πίνακα:

Επίπεδο Θορύβου	Μέθοδος (Προσέγγιση)	Βέλτιστο σ	Βέλτιστο θ_{edge}	Μέγιστο Κριτήριο C
PSNR = 20 dB (I_1)	Γραμμική (LoG)	1.24	0.21	0.894
	Μη-Γραμμική	1.24	0.18	0.913
PSNR = 10 dB (I_2)	Γραμμική (LoG)	1.97	0.25	0.731
	Μη-Γραμμική	2.16	0.23	0.767

Συγκεντρωτικός πίνακας βέλτιστων παραμέτρων και απόδοσης (Grid Search) του αλγορίθμου EdgeDetect.

Συμπεράσματα και Ερμηνεία Αποτελεσμάτων

Η ανάλυση των παραπάνω αποτελεσμάτων, σε συνδυασμό με την οπτική αξιολόγηση των παραγόμενων edge maps, μας οδηγεί σε συμπεράσματα σχετικά με τη συμπεριφορά του αλγορίθμου για edge detection:

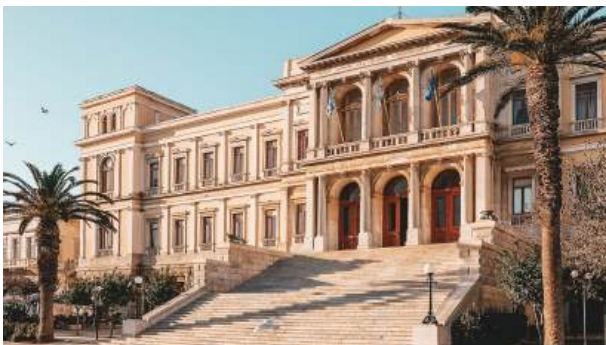
- **Επίδραση του PSNR:** Όπως ήταν αναμενόμενο, η υψηλότερη τιμή PSNR (20 dB) οδήγησε σε καλύτερα αποτελέσματα (μεγαλύτερο κριτήριο C) σε σχέση με τα 10 dB. Η αύξηση της διασποράς του θορύβου "πνίγει" την υψίσυχη πληροφορία των πραγματικών ακμών, καθιστώντας τον διαχωρισμό τους από το θόρυβο πολύ δύσκολο. Επιπλέον, επιβεβαιώνεται ότι η μετρική C αποτελεί ένα εξαιρετικά αξιόπιστο ποσοτικό κριτήριο, καθώς οι αυξομειώσεις της ταυτίζονται με την οπτική ποιότητα των εικόνων, που βλέπαμε στα πειράματά μας.
- **Η Συμπεριφορά της Εξομάλυνσης (σ):** Η παράμετρος σ ελέγχει το trade-off μεταξύ μείωσης του θορύβου (τον κάνει blend ώστε να χαθεί) και διατήρησης λεπτομερειών. Θεωρητικά, καθώς το $\sigma \rightarrow 0$, η Γκαουσιανή συνάρτηση τείνει στη συνάρτηση δέλτα του Dirac ($\delta(x, y)$), δηλαδή το φίλτρο επειδή εφαρμόζεται με συνέλιξη, δεν κάνει πρακτικά τίποτα στην εικόνα εισόδου. Σε αυτή την περίπτωση, διατηρούνται όλες οι πληροφορίες, αλλά ο αλγόριθμος έχει πρόβλημα με τον θόρυβο. Αντίθετα, για μεγάλα σ , η εικόνα υφίσταται έντονο blend, "σβήνοντας" την πληροφορία. Αυτό εξηγεί γιατί στην εικόνα των 10 dB ο αλγόριθμος "επέλεξε" βέλτιστο $\sigma \approx 2.0$ (έναντι ~ 1.24 στα 20 dB): ο έντονος θόρυβος επέβαλε ισχυρότερη εξομάλυνση, με κόστος όμως την απώλεια ορισμένων λεπτών ακμών.
- **Η Συμπεριφορά της Κατωφλίωσης (θ_{edge}):** Το κατώφλι καθορίζει την "αυστηρότητα" του αλγορίθμου. Για πολύ μικρές τιμές, ο αλγόριθμος εμφανίζει υψηλό Recall αλλά χαμηλό Precision, καθώς κατηγοριοποιεί εσφαλμένα τις διακυμάνσεις του θορύβου ως ακμές. Για υπερβολικά μεγάλες τιμές, συμβαίνει ακριβώς το ανάποδο. Αξίζει να επισημάνουμε ότι η βέλτιστη τιμή του θ_{edge} παρέμεινε σχετικά σταθερή (γύρω στο 0.20 – 0.25) ανεξαρτήτως θορύβου. Αυτό συμβαίνει διότι το κατώφλι εξαρτάται πρωτίστως από την ενδογενή "contrast" (αντιθέση) των αντικειμένων της αρχικής εικόνας.
- **Σύγκριση Γραμμικής (LoG) vs. Μη-Γραμμικής Προσέγγισης:** Σταθερά, σε όλα τα πειράματα, η μη-γραμμική προσέγγιση υπερτερούσε της γραμμικής. Το γραμμικό φίλτρο (συνέλιξη) αποτελεί ουσιαστικά έναν σταθμισμένο μέσο όρο. Όταν εφαρμόζεται σε μια απότομη ακμή, τείνει να διαχέει την ενέργειά της στα γειτονικά pixels, υποβαθμίζοντας τη χωρική ακρίβεια της ακμής (που ακριβώς δηλαδή βρίσκεται). Αντιθέτως, τα μη-γραμμικά μορφολογικά φίλτρα βασίζονται σε τελεστές ακρότατων ($\min/\max$ μέσω διάβρωσης/διαστολής). Αυτό τα καθιστά εγγενώς πιο robust απέναντι σε μεμονωμένα outliers, επιτρέποντάς τους να διατηρούν την οξύτητα και την ακριβή θέση των πραγματικών ακμών με μεγαλύτερη επιτυχία.

1.4 Εφαρμογή των Αλγορίθμων Ανίχνευσης Ακμών σε Πραγματικές Εικόνες

Στο τελευταίο στάδιο του πρώτου μέρους, κληθήκαμε να αξιολογήσουμε τη γενίκευση και την αξιοπιστία του αλγορίθμου μας, εφαρμόζοντάς τον σε μία νέα, πιο σύνθετη εικόνα, καθώς είναι πραγματική φωτογραφία.

1.4.1 Προετοιμασία Δεδομένων και Ground Truth

Ακολουθώντας την ίδια μεθοδολογία με τα προηγούμενα ερωτήματα, αρχικά φορτώσαμε την έγχρωμη εικόνα και τη μετατρέψαμε σε grayscale. Έπειτα, για να έχουμε το μέτρο σύγκρισης για την αξιολόγηση των αποτελεσμάτων μας (που δε θα μας χρειάσει ιδιαίτερα εδώ βέβαια), δημιουργήσαμε την εικόνα των πραγματικών ακμών (Ground Truth) χρησιμοποιώντας τον μορφολογικό τελεστή που περιγράφηκε στην ενότητα 1.3.1 (με την παράμετρο θ_{edge} πειραματικά).



Αρχική Εικόνα (View Image)



Grayscale & Αληθινές Ακμές

1.4.2 Βελτιστοποίηση Οπτικών Αποτελεσμάτων - Ποιοτική Αξιολόγηση

Για να εντοπίσουμε τον ιδανικό συνδυασμό παραμέτρων, αξιολογώντας ποιοτικά, που αναδεικνύει τις πραγματικές ακμές της εικόνας χωρίς να έχουμε άχρηστες λεπτομέρειες, ακολουθήσαμε μια συστηματική προσέγγιση. Επιλέξαμε 5 αντιπροσωπευτικές τιμές για το κατώφλι θ_{edge} (π.χ. 0.10, 0.15, 0.18, 0.20, 0.25) και, για κάθε μία από αυτές, δοκιμάσαμε 4 διαφορετικές κλίμακες εξομάλυνσης σ (π.χ. 0.5, 1.0, 1.5, 2.0).

Εκτελώντας τον αλγόριθμο, χρησιμοποιώντας τη μη-γραμμική προσέγγιση της Λαπλασιανής διότι είχε συνολικά καλύτερα αποτελέσματα, τυπώσαμε συνολικά 20 δυαδικούς edge maps. Μέσα από τη διεξοδική οπτική σύγκριση αυτών των παραγόμενων εικόνων, καταλήξαμε στο βέλτιστο (υποκειμενικά) αποτέλεσμα.



Τα Πειράματά μας με διαφορετικές τιμές των παραμέτρων σ και θ_{edge} .

Το Βέλτιστο Αποτέλεσμα:

Το πιο ισορροπημένο οπτικά αποτέλεσμα επιτεύχθηκε για τις παραμέτρους $\sigma = 2$ και $\theta_{edge} = 0.2$.

Εστίασαμε στο να μην ανιχνεύονται οι μικρές υφές (όπως τα φύλλα των δέντρων ή οι λεπτομέρειες του δρόμου), αλλά να διατηρούνται οι κύριες δομικές ακμές (όπως τα περιγράμματα των κτιρίων, τα παράθυρα και ο ορίζοντας). Η επιλογή αυτών των παραμέτρων επέτρεψε στον αλγόριθμο να καταστείλει αποτελεσματικά τον θόρυβο και τις ασήμαντες λεπτομέρειες,

ενώ ταυτόχρονα διατήρησε την οξύτητα και την ακρίβεια των πραγματικών ακμών της εικόνας.

Best Edge Map (Non-Linear) - Sigma: 2.0, Theta: 0.2



Το βέλτιστο αποτέλεσμα ανίχνευσης ακμών ($\sigma = 2.0, \theta_{edge} = 0.2$).

Συνολικά, μπορούμε να διαπιστώσουμε με αυτήν την περίπλοκη εικόνα ότι ο αλγόριθμος που έχουμε κατασκευάσει είναι αρκετά αξιόπιστος στην ανίχνευση ακμών. Παρά το γεγονός ότι η εικόνα περιέχει πολλές λεπτομέρειες και υφές, ο αλγόριθμος κατάφερε να διατηρήσει τις σημαντικές δομικές ακμές, αποφεύγοντας την υπερβολική ανίχνευση που θα μπορούσε να προκύψει από τον θόρυβο ή τις μικρές υφές. Αυτό επιβεβαιώνει την αποτελεσματικότητα της προσέγγισης μας, ειδικά όταν χρησιμοποιούνται οι βέλτιστες παράμετροι για κάθε συγκεκριμένη εικόνα.

Παρατηρούμε ότι η αύξηση της παραμέτρου σ , στην περίπτωση μας, βοηθάει με το να "διώξουμε" τις μικρές υφές και να διατηρήσουμε μόνο τις σημαντικές ακμές. Επιπλέον φαίνεται η μείωση της παραμέτρου αυτής να μην επηρεάζει σημαντικά την απόδοση, καθώς η εικόνα δεν έχει θόρυβο που θα δημιουργούσε πρόβλημα σε αυτήν την περίπτωση, ωστόσο αυξάνονται οι ανεπιθύμητες λεπτομέρειες που ανιχνεύονται.

Όσον αφορά την παράμετρο κατωφλίου, παρατηρούμε ότι η αύξηση της αυστηρότητας (αύξηση του θ_{edge}) οδηγεί σε πιο "καθαρά" αποτελέσματα, με λιγότερες ψευδείς ακμές, αλλά ταυτόχρονα υπάρχει κίνδυνος να χαθούν σημαντικές ακμές αν το κατώφλι γίνει υπερβολικά υψηλό. Στην περίπτωση μας, το $\theta_{edge} = 0.2$ φαίνεται να προσφέρει αυτήν την ισορροπία.

Μέρος 2: Ανίχνευση Σημείων Ενδιαφέροντος (Interest Point Detection)

Στο 2^ο μέρος του Project θα ασχοληθούμε όπως αναφέρει και ο τίτλος, με την ανίχνευση σημείων ενδιαφέροντος. Με τον όρο σημεία ενδιαφέροντος, αναφερόμαστε στις θέσεις της εικόνας όπου τα αντίστοιχα pixels διαφέρουν από τα υπόλοιπα της γειτονιάς τους, δηλαδή έχουν κάτι ξεχωριστό. Τέτοια σημεία είναι συνήθως οι γωνίες των αντικειμένων μιας εικόνας, τεμνόμενες ακμές ή και σημεία με έντονη υφή ή κάποιο μοτίβο.

2.1. Ανίχνευση Ακμών

Στο πρώτο ερώτημα, θα εργαστούμε πάνω στην ανίχνευση ακμών μέσω της μεθόδου των Harris-Stephens.

Η μέθοδος Harris-Stephens αποτελεί έναν αλγόριθμο εύρεσης γωνιών σε εικόνες, δηλαδή σημείων όπου η φωτεινότητα αλλάζει απότομα προς όλες τις κατευθύνσεις. Η ιδιαιτερότητα του συγκεκριμένου αλγορίθμου έναντι άλλων τρόπων εύρεσης γωνιών, έγκειται στη χρήση ενός μαθηματικού πίνακα, γνωστού και ως πίνακα Harris (δομικός τανυστής J), ο οποίος ποσοτικοποιεί τις τοπικές μεταβολές της φωτεινότητας. Αντί να μετακινεί δοκιμαστικά ένα παράθυρο γύρω από κάθε pixel, ο αλγόριθμος υπολογίζει ένα μέτρο "γωνιότητας" από τις ιδιοτιμές αυτού του πίνακα. Με βάση αυτό, κάθε σημείο ταξινομείται είτε ως γωνία, είτε ως ακμή (όπου η αλλαγή γίνεται μόνο προς μία κατεύθυνση), είτε ως επίπεδη περιοχή.

Ο δομικός τανυστής J , μας μιλάει για την πληροφορία της κλίσης μέσα σε μια γειτονιά του κάθε pixel. Είναι ένας συμμετρικός πίνακας, 3^{ων} στοιχείων J_1, J_2, J_3 , τα οποία στη συγκεκριμένη άσκηση υπολογίζονται συναρτήσει της εικόνας I_σ , η οποία αποτελεί μια ομαλοποιημένη εκδοχή της αρχικής εικόνας I με Gaussian κλίμακας σ , στην οποία στη συνέχεια εφαρμόζεται και μια δεύτερη Gaussian συνέλιξης κλίμακας ρ .

Σαν πρώτο βήμα, πρέπει να βρούμε το μέγεθος του κάθε Gaussian πυρήνα. Σε μια Gaussian κατανομή, γνωρίζουμε ότι:

- 68.3% των τιμών βρίσκονται στο διάστημα $[-\sigma, \sigma]$
- 95.4% των τιμών βρίσκονται στο διάστημα $[-2\sigma, 2\sigma]$
- 99.7% των τιμών βρίσκονται στο διάστημα $[-2\sigma, 2\sigma]$

Έτσι, επιλέγουμε το διάστημα $[-3\sigma, 3\sigma]$ αφού περιέχει σχεδόν όλη την πληροφορία, με τις τιμές εκτός αυτού να είναι πρακτικά μηδενικές, με αποτέλεσμα να μην επηρεάζουν το αποτέλεσμα της συνέλιξης. Οπότε το μέγεθος του πυρήνα θα είναι:

$$N = \lceil 3\sigma \rceil \cdot 2 + 1$$

2.1.1. Υπολογισμός Τανυστή J

Στο πρώτο υποερώτημα, επιθυμούμε να βρούμε τον δομικό τανυστή J στον οποίον αναφερθήκαμε προηγουμένως και δίνεται από τους παρακάτω τύπους:

- $J_1(x, y) = G_\rho * \left(\frac{\partial I_\sigma}{\partial x} \cdot \frac{\partial I_\sigma}{\partial x} \right) (x, y)$
- $J_2(x, y) = G_\rho * \left(\frac{\partial I_\sigma}{\partial x} \cdot \frac{\partial I_\sigma}{\partial y} \right) (x, y)$
- $J_3(x, y) = G_\rho * \left(\frac{\partial I_\sigma}{\partial y} \cdot \frac{\partial I_\sigma}{\partial y} \right) (x, y)$

όπου $I_\sigma = G_\rho * I$, με τους G_ρ, G_σ να είναι δισδιάστατοι Gaussian πυρήνες.

Ο δομικός τανυστής J , όπως μπορούμε να δούμε και παραπάνω, δεν είναι απλώς η τιμή της παραγώγου σε ένα μεμονωμένο pixel, αλλά επιχειρεί να περιγράψει τη συμπεριφορά της κλίσης μέσα σε μια ευρύτερη γειτονιά, ώστε να εξάγει ένα αξιόπιστο συμπέρασμα για την τοπική δομή της εικόνας.

Για αυτόν το λόγο, κατά τον υπολογισμό του, μπορούμε να διακρίνουμε δύο ξεχωριστά στάδια εξομάλυνσης:

- Αρχικά, εφαρμόζεται ο πυρήνας G_σ στην αρχική εικόνα I με σκοπό την μείωση της επίδρασης του θορύβου. Σε αντίθετη περίπτωση, ο θόρυβος της εικόνας θα μπορούσε να οδηγήσει σε λανθασμένα αποτελέσματα λόγω της παραγωγίσιμης που εφαρμόζεται.

- Στη συνέχεια, εφαρμόζεται ο πυρήνας G_p ώστε να **ενσωματώσει την πληροφορία των γινομένων από μια ευρύτερη περιοχή**. Έτσι, μπορούμε να προλάβουμε λανθασμένα αποτελέσματα που θα προέκυπταν αν βασιζόμασταν σε ένα μόνο pixel, το οποίο μπορεί να επηρεάζεται από θόρυβο ή να μην είναι αντιπροσωπευτικό της γειτονιάς του. Η ολοκλήρωση αυτή ουσιαστικά υπολογίζει τον τοπικό μέσο όρο των γινομένων, δίνοντας μας μια πιο σταθερή και αξιόπιστη περιγραφή της δομής γύρω από κάθε σημείο, κάνοντας έτσι πιο ακριβή το διαχωρισμό των εικόνων σε επίπεδες περιοχές, ακμές και γωνίες.

2.1.2. Υπολογισμός ιδιοτιμών λ_- , λ_+ του τανυστή J

Στο επόμενο βήμα, θέλουμε να υπολογίσουμε τις ιδιοτιμές λ_- , λ_+ του πίνακα J , σύμφωνα με τη σχέση:

$$\lambda_{\pm}(x, y) = \frac{1}{2} \cdot (J_1 + J_3 \pm \sqrt{(J_1 - J_3)^2 + 4 \cdot J_2^2})$$

Στο παρακάτω σχήμα παρουσιάζονται οι ιδιοτιμές του δομικού τανυστή για την εικόνα "solar.jpg", με παραμέτρους $\sigma = 2$ και $p = 2.5$.



Οι εικόνες των ιδιοτιμών λ_+ και λ_- επιβεβαιώνουν τη θεωρητική ερμηνεία του δομικού τανυστή. Η λ_+ αντιστοιχεί στη μέγιστη κατεύθυνση μεταβολής της φωτεινότητας, δηλαδή παίρνει μεγάλες τιμές σε ακμές και γωνίες, αναδεικνύοντας τα όρια και τις δομές των αντικειμένων, ενώ η λ_- αντιστοιχεί στην ελάχιστη κατεύθυνση μεταβολής, δηλαδή παραμένει μικρή σε ακμές (όπου μια κατεύθυνση έχει μηδενική μεταβολή) και παίρνει μεγαλύτερες τιμές μόνο σε γωνίες, όπου η φωτεινότητα αλλάζει έντονα και στις δύο κατευθύνσεις ταυτόχρονα.

Υλοποιώντας το Project, μας γεννήθηκε η απορία του τι γίνεται όταν **αλλάζουμε τα σ, p** .

Πιο συγκεκριμένα, για $\sigma = 4$ και $p = 2.5$, δηλαδή αυξάνοντας το σ και κρατώντας σταθερό το p λαμβάνουμε:



Αυξάνοντας την κλίμακα διαφύση σε $\sigma = 4$, το ισχυρό blur πριν τον υπολογισμό των παραγώγων **εξαλείφει και τις μικρές λεπτομέρειες παράλληλα με τον θόρυβο**. Έτσι, τα λ_+ , λ_- παίρνουν μεγάλες τιμές μόνο σε έντονες δομές της εικόνας, όπως μεγάλες γωνίες και ακμές, ενώ μικρές γωνίες και λεπτομέρειες που υπήρχαν στην εικόνα με $\sigma = 2$ έχουν πλέον εξομαλυνθεί και δεν ανιχνεύονται.

Αντίθετα, αν αυξήσουμε τώρα πολύ το p , δηλαδή την κλίμακα ολοκλήρωσης, λαμβάνουμε:



Με μεγάλη τώρα κλίμακα ολοκλήρωσης $p = 4$, το Gaussian blur εφαρμόζεται στα γινόμενα των παραγώγων πάνω σε πιο εκτεταμένη γειτονιά, με αποτέλεσμα η πληροφορία των γειτονικών pixels να συσσωρεύεται. Έτσι, το λ_- που με μικρό p ήταν σχεδόν μηδενικό παντού (μαύρη εικόνα) αποκτά πλέον σημαντικές τιμές σε ευρύτερες περιοχές γύρω από τις γωνίες. Αυτό σημαίνει ότι ο **ανιχνευτής γίνεται πιο ευαίσθητος αλλά χάνει παράλληλα και χωρική**

ακρίβεια. Με άλλα λόγια, δεν μπορεί να εντοπίσει με ακρίβεια πού βρίσκεται ακριβώς μια γωνία, μόνο ότι υπάρχει κάπου στην περιοχή.

2.1.3. Κριτήριο γωνιότητας R

Αφού βρούμε τις ιδιοτιμές του πίνακα μας, χρειαζόμαστε προφανώς και ένα κριτήριο ώστε να διακρίνουμε αν ένα pixel εμπεριέχει ή όχι κάποια γωνία.

Για να το κάνουμε αυτό, θα χρησιμοποιήσουμε το κριτήριο γωνιότητας:

$$R(x, y) = \lambda_- \lambda_+ - k \cdot (\lambda_- + \lambda_+)^2$$

όπου k μικρή θετική σταθερά η οποία καθορίζει την ευαισθησία του ανιχνευτή.

Τέλος, αφού ορίσαμε το κριτήριο μας, θα λέμε ότι **ένα pixel αναπαριστά γωνία όταν:**

- Είναι **μέγιστο του R** εντός τετραγωνικού παραθύρου που το περιβάλλει, το μέγεθος του οποίου εξαρτάται από την κλίμακα σ .
- Αντιστοιχεί σε τιμή του **R μεγαλύτερη από ένα ποσοστό του ολικού μεγίστου του**, δηλαδή $R(x, y) > \theta_{corn} \cdot R_{max}$, όπου θ_{corn} ένα κατάλληλα επιλεγμένο κατώφλι.

Ουσιαστικά, το R παίρνει **θετικές τιμές στις γωνίες, αρνητικές ή σχεδόν μηδενικές στις ακμές και μικρές τιμές στις επίπεδες περιοχές.**

Για την αρχική εφαρμογή του αλγορίθμου, επιλέχθηκαν οι παρακάτω παράμετροι:

- $\sigma = 2$: Τυπική απόκλιση του Gaussian για την εξομάλυνση πριν τον υπολογισμό των παραγώγων, επιλέχθηκε ώστε να μειώσει τον θόρυβο χωρίς να εξαλείψει σημαντικές λεπτομέρειες.
- $p = 2.5$: Τυπική απόκλιση του Gaussian για την ολοκλήρωση των γινομένων των παραγώγων, επιλέχθηκε ώστε να ενσωματώσει πληροφορία από μια μικρή γειτονιά, διατηρώντας παράλληλα καλή χωρική ακρίβεια.
- $k = 0.05$: Σταθερά που καθορίζει την ευαισθησία του ανιχνευτή, επιλέχθηκε μέσα στο συνιστώμενο εύρος (0.04-0.06) για να επιτύχει καλή ισορροπία μεταξύ ανίχνευσης γωνιών και αποφυγής ψευδών θετικών.
- $\theta_{corn} = 0.005$: Κατώφλι για την επιλογή των γωνιών, επιλέχθηκε ώστε να διατηρήσει μόνο τα πιο έντονα σημεία ως γωνίες, απορρίπτοντας τα λιγότερο σημαντικά.

Έτσι, τα αποτελέσματα που λαμβάνουμε είναι τα εξής:

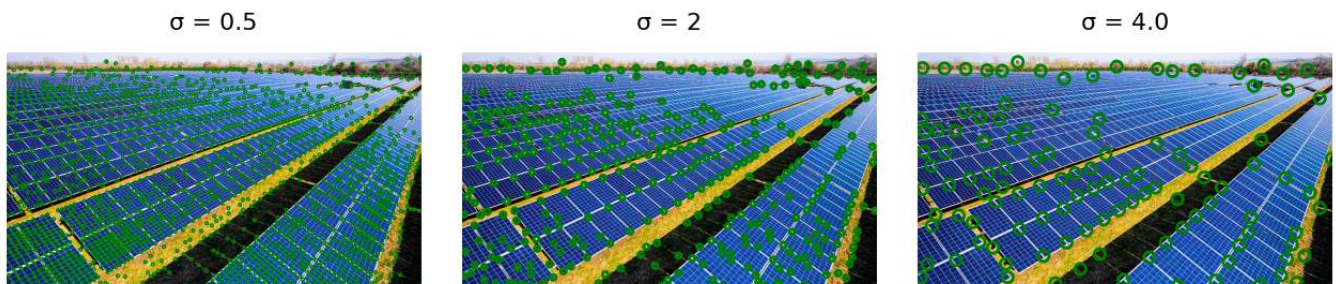


Παρατηρώντας τις παραπάνω εικόνες, βλέπουμε ότι όντως έχουν ανιχνευθεί σωστά οι γωνίες τους, με τα αποτελέσματα πάντα να εξαρτώνται από την επιλογή των παραμέτρων μας.

Πιο συγκεκριμένα, εφαρμόσαμε τον αλγόριθμο εντοπισμού γωνιών στην αρχική εικόνα "solar.jpg" και σε αυτή που προέκυψε μετά την ανίχνευση ακμών του μέρους 1.

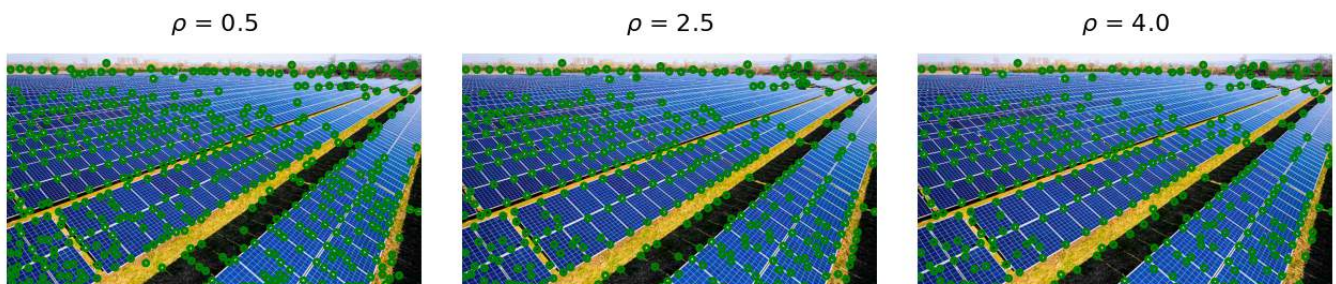
Θεωρητικά, θα περιμέναμε **παρόμοια σημεία ενδιαφέροντος**, καθώς ο εντοπισμός ακμών κρατάει και τις γωνίες. Ωστόσο, τα αποτελέσματα είναι διαφορετικά, αφού στην πραγματικότητα η διαδικασία ανίχνευσης ακμών **αλλοιώνει την αρχική πληροφορία της εικόνας**. Συγκεκριμένα, αφαιρεί μεγάλο μέρος της πληροφορίας έντασης και εξασθενεί τις τοπικές μεταβολές που χρησιμοποιεί ο αλγόριθμος για να εντοπίσει γωνίες. Έτσι, ο Harris detector υπολογίζει διαφορετικές ιδιοτιμές και τελικά εντοπίζει διαφορετικά σημεία.

Αν τώρα **αλλάξουμε την κλίμακα διαφόρισης σ** , θα δούμε ότι με μεγαλύτερο σ ανιχνεύονται μόνο οι πιο έντονες γωνίες, ενώ με μικρότερο σ ανιχνεύονται και πιο αμυδρές, αλλά παράλληλα αυξάνεται και η ανίχνευση θορύβου.



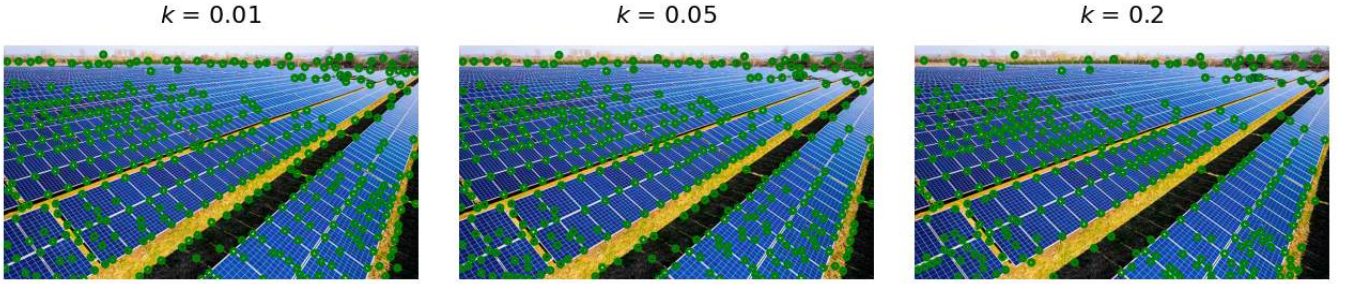
Ένα σχόλιο που μπορούμε επίσης να κάνουμε, είναι ότι η **ακτίνα των πράσινων κύκλων εξαρτάται από το σ** , άρα για μεγαλύτερο σ θα έχουμε μεγαλύτερους κύκλους, αφού η γωνία που ανιχνεύουμε είναι πιο θολή και δεν μπορούμε να προσδιορίσουμε με ακρίβεια τη θέση της, ενώ για μικρότερο θα έχουμε μικρότερους κύκλους, αφού η γωνία είναι πιο έντονη και μπορούμε να προσδιορίσουμε με μεγαλύτερη ακρίβεια τη θέση της.

Στη συνέχεια, για **διαφορετικές τιμές του ρ** , περιμένουμε να δούμε ότι για μεγαλύτερο ρ θα ανιχνεύονται λιγότερες γωνίες, καθώς η ομαλοποίηση σε μεγαλύτερη γειτονία εξαλείφει μικρότερες διακυμάνσεις, ενώ με μικρότερο ρ αναμένουμε να ανιχνεύονται περισσότερες γωνίες, αλλά με μεγαλύτερη πιθανότητα να είναι ψευδείς, καθώς βασιζόμαστε σε λιγότερη πληροφορία για κάθε pixel.

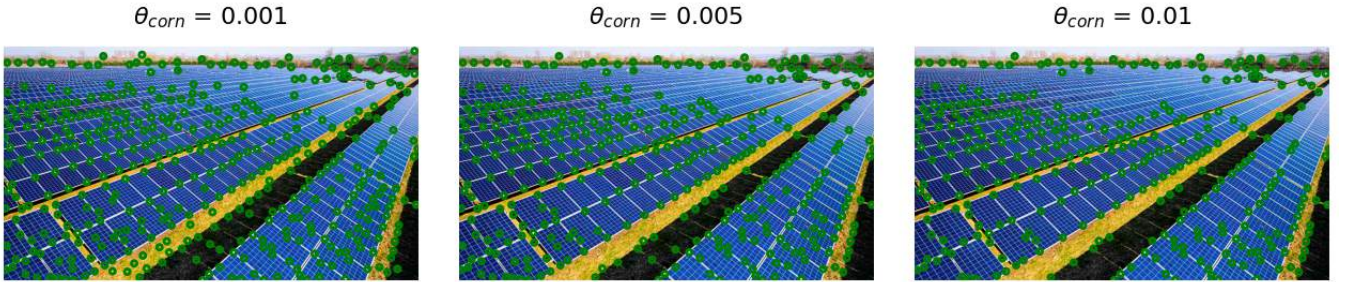


Παρατηρώντας τις παραπάνω εικόνες, βλέπουμε ότι πράγματι επαληθεύονται οι εκτιμήσεις μας.

Για διαφορετικές τώρα τιμές του k , αναμένουμε ότι για μεγαλύτερο k θα ανιχνεύονται λιγότερες γωνίες, καθώς θα αυξάνεται ο δεύτερος όρος του κριτηρίου με αποτέλεσμα να απορρίπτονται περισσότερα σημεία, ενώ για μικρότερο k , το κριτήριο μας θα παίρνει πιο εύκολα θετικές τιμές, με αποτέλεσμα να ανιχνεύονται περισσότερες γωνίες, αλλά με μεγαλύτερη πιθανότητα να είναι ψευδείς.



Τέλος, αξία έχει και η οπτικοποίηση των διαφορών για διαφορετικές τιμές του θ_{corn} , αφού όσο πιο χαμηλό είναι το κατώφλι, τόσο περισσότερες γωνίες ανιχνεύονται, αλλά παράλληλα αυξάνεται και ο αριθμός των ψευδών θετικών. Αντίθετα, όσο πιο υψηλό είναι το κατώφλι, τόσο λιγότερες γωνίες ανιχνεύονται, αλλά με μεγαλύτερη αξιοπιστία και ακρίβεια.



2.2. Πολυκλιμακωτή Ανίχνευση Γωνιών

Στο προηγούμενο ερώτημα (2.1), η ανίχνευση γωνιών πραγματοποιούνταν σε μία μόνο κλίμακα, η οποία καθοριζόταν από τις παραμέτρους σ (κλίμακα διαφύρισης) και ρ (κλίμακα ολοκλήρωσης). Αυτό σημαίνει ότι ο ανιχνευτής που φτιάξαμε αναγνώριζε γωνίες συγκεκριμένου μεγέθους, ανάλογα με την επιλογή αυτών των παραμέτρων.

Ωστόσο, σε φυσικές εικόνες, τα αντικείμενα, και άρα και οι γωνίες τους, εμφανίζονται σε διαφορετικά μεγέθη. Για να μπορέσουμε λοιπόν να ανιχνεύσουμε όλες τις γωνίες ανεξάρτητα από το μέγεθος τους, **χρειαζόμαστε μια προσέγγιση πολλαπλών κλιμάκων.**

Στο ερώτημα αυτό, θα επεκτείνουμε τη μέθοδο Harris-Stephens ώστε να λειτουργεί σε πολλαπλές κλίμακες, ακολουθώντας τη λογική του Harris-Laplacian.

Η μέθοδος αποτελείται από δύο στάδια:

- **Επιλογή υποψήφιων σημείων σε κάθε κλίμακα:** Για κάθε κλίμακα (διαφορετικές τιμές σ και ρ), εφαρμόζουμε τον ανιχνευτή Harris όπως στο 2.1 και εντοπίζουμε τις αντίστοιχες γωνίες.
- **Τελική επιλογή σημείων με βάση το μέγιστο στην κλίμακα:** Από όλες τις υποψήφιες γωνίες, κρατάμε μόνο εκείνες που παρουσιάζουν μέγιστη απόκριση σε κάποια μετρική αμετάβλητη ως προς την κλίμακα (π.χ. Laplacian-of-Gaussian όπως θα δούμε και παρακάτω). Έτσι, για κάθε γωνία, γνωρίζουμε όχι μόνο τη θέση της (x, y) αλλά και την κλίμακα στην οποία ανιχνεύθηκε βέλτιστα.

2.2.1. Εφαρμογή ανιχνευτή Harris για διαφορετικές κλίμακες

Πιο συγκεκριμένα, στο πρώτο στάδιο θα επαναλάβουμε τη διαδικασία που κάναμε πριν, αλλά τώρα για N κλίμακες με:

$$\begin{aligned}\sigma_0, \sigma_1, \dots, \sigma_{N-1} &= s^0 \sigma_0, s^1 \sigma_1, \dots, s^{N-1} \sigma_{N-1} \\ \rho_0, \rho_1, \dots, \rho_{N-1} &= s^0 \rho_0, s^1 \rho_1, \dots, s^{N-1} \rho_{N-1}\end{aligned}$$

όπου σ_0, ρ_0 οι αρχικές κλίμακες και s ο παράγοντας κλιμάκωσης.

Μια παρατήρηση που μπορούμε να κάνουμε σε αυτό το σημείο είναι ότι εφαρμόζουμε **γεωμετρική κλιμάκωση** αντί για γραμμική ή κάποια άλλη, αφού ανταποκρίνεται καλύτερα στον τρόπο που αντιλαμβανόμαστε τα μεγέθη, καλύπτοντας ομοιόμορφα το φάσμα των κλιμάκων με λιγότερες επαναλήψεις.

2.2.2. Επιλογή Χαρακτηριστικής Κλίμακας

Αφού εφαρμόσαμε τον ανιχνευτή μας σε πολλαπλές κλίμακες, πρέπει τώρα να επιλέξουμε για κάθε γωνία την κλίμακα στην οποία έχει ανιχνευθεί με την μεγαλύτερη ένταση. Έτσι, ενώ μια γωνία είναι πολύ πιθανό να είναι ορατή σε παραπάνω από μία κλίμακες, εμείς επιθυμούμε να κρατήσουμε μόνο **μία εμφάνιση ανά γωνία, δηλαδή εκείνη που αντιστοιχεί στη χαρακτηριστική της κλίμακα**, όπου δηλαδή η δομή της αναδεικνύεται με τον βέλτιστο τρόπο.

Για να την εντοπίσουμε, χρησιμοποιούμε την **κανονικοποιημένη Λαπλασιανή της Γκαουσιανής (LoG)**, η οποία μετράει πόσο έντονη είναι η αλλαγή της φωτεινότητας σε κάθε κλίμακα και ορίζεται ως:

$$|LoG(x, \sigma_i)| = \sigma_i^2 \cdot |L_{xx}(x, \sigma_i) + L_{yy}(x, \sigma_i)|, \quad i = 0, \dots, N - 1$$

όπου L_{xx}, L_{yy} οι δεύτερες παράγωγοι της εξομαλυμένης εικόνας.

Τέλος, ο πολλαπλασιασμός με σ^2 λειτουργεί ως **παράγοντας διόρθωσης**: όσο πιο μεγάλη είναι η κλίμακα (άρα πιο θολή η εικόνα), τόσο μικρότερες γίνονται οι τιμές των παραγώγων, αφού η ίδια αλλαγή της εικόνας απλώνεται σε μεγαλύτερη περιοχή, οπότε οι κλίσεις γίνονται πιο ήπιες. Το σ^2 αντισταθμίζει αυτήν τη μείωση, επιτρέποντας τη σύγκριση αποκρίσεων μεταξύ διαφορετικών κλιμάκων.

Αφού βρούμε κάθε υποψήφια γωνία σε κάθε κλίμακα ξεχωριστά, θέλουμε τώρα να αναγνωρίσουμε για κάθε μία από αυτές σε ποια κλίμακα εμφανίζεται πιο έντονα.

Για να το πετύχουμε αυτό, εφαρμόζουμε το εξής κριτήριο: **για κάθε υποψήφια γωνία που εντοπίστηκε στην κλίμακα i** , συγκρίνουμε την τιμή της κανονικοποιημένης LoG στην κλίμακα αυτή, με τις αντίστοιχες τιμές στις δύο **γειτονικές κλίμακες** ($i + 1$ και $i - 1$). Έτσι, η γωνία παραμένει μόνο αν:

$$LoG_i(x, y) > LoG_{i+1}(x, y) \quad \text{και} \quad LoG_i(x, y) > LoG_{i-1}(x, y)$$

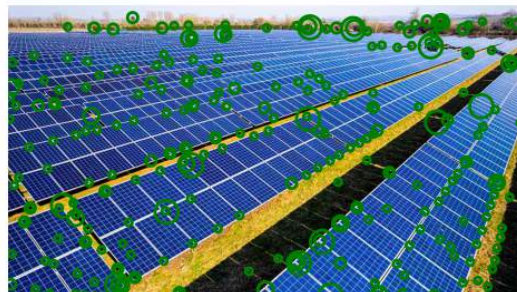
Με άλλα λόγια, απαιτούμε η τιμή στην τρέχουσα κλίμακα να είναι μεγαλύτερη και από τις δύο γειτονικές. Άρα η συγκεκριμένη κλίμακα αποτελεί τοπικό μέγιστο στο χώρο των κλιμάκων για τη συγκεκριμένη γωνία, οπότε είναι η χαρακτηριστική της κλίμακα. Αν η συνθήκη δεν ικανοποιείται, απορρίπτουμε τη γωνία.

Σημείωση: Ελέγχουμε μόνο τις δύο γειτονικές κλίμακες γιατί η LoG μεταβάλλεται ομαλά καθώς αλλάζουμε κλίμακα.

Οι παράμετροι που χρησιμοποιήθηκαν για την πολυκλιμακωτή ανίχνευση ήταν ακριβώς οι ίδιοι με αυτούς της ενότητας 2.1, με τις πρόσθετες μεταβλητές να είναι $N = 4$ (αριθμός κλιμάκων) και $s = 1.5$ (παράγοντας κλιμάκωσης).

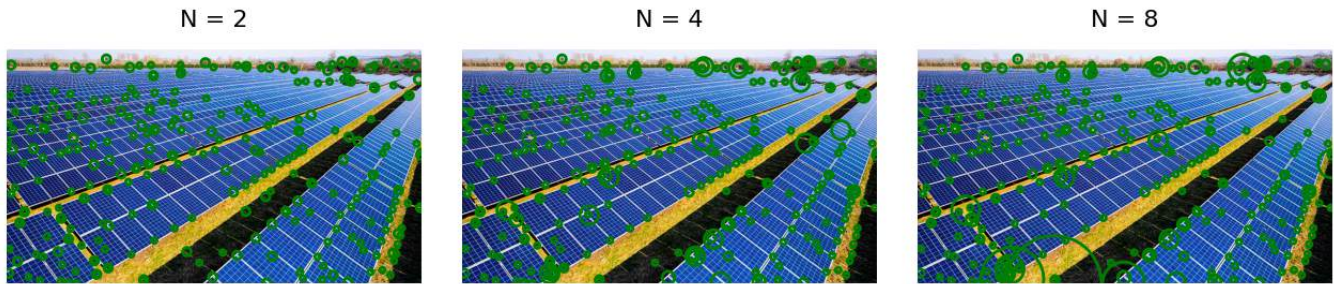
Τα αποτελέσματα που λαμβάνουμε είναι τα εξής:

solar



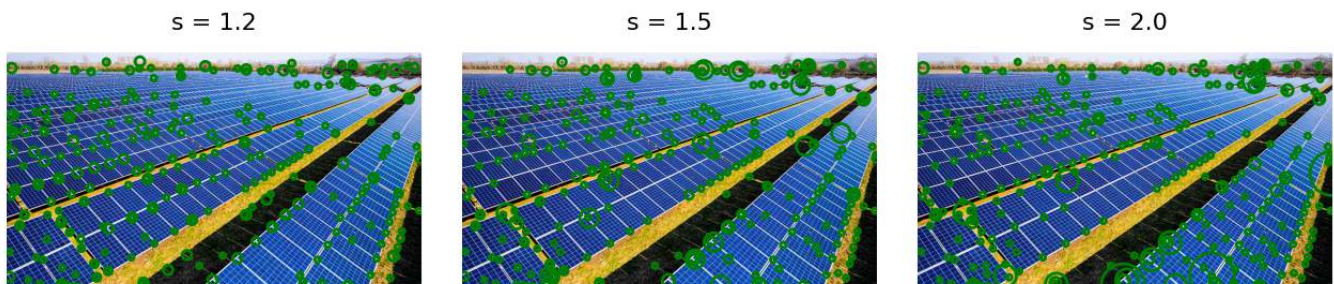
Από την παραπάνω εικόνα, παρατηρούμε ότι σε **περιοχές με πιο λεπτές λεπτομέρειες**, όπως πχ στις ενδιάμεσες ενώσεις των ηλιακών πάνελ, οι **κύκλοι έχουν μικρή ακτίνα**, κάτι που υποδηλώνει μικρές χαρακτηριστικές κλίμακες. Αντίθετα, σε **μεγάλες δομές ή επιφάνειες με έντονες γωνίες**, οι **κύκλοι είναι μεγαλύτεροι**, δηλαδή αντιστοιχούν σε μεγαλύτερες κλίμακες παρατήρησης. Βλέπουμε λοιπόν πως με τη μέθοδο αυτή προκύπτουν πιο ακριβή αποτελέσματα που ανταποκρίνονται καλύτερα στις πραγματικές γωνίες των εικόνων.

Όμοια με πριν, έχει και εδώ αξία να δούμε πως επηρεάζουν τα αποτελέσματα οι εξτρά παράμετροι που εισάγαμε. Σε πρώτη φάση, για διαφορετικές τιμές του N , θα λάβουμε:



Οι κύριες παρατηρήσεις που μπορούμε να κάνουμε είναι δύο. Αρχικά, για μικρό N , οι κύκλοι έχουν παρόμοια μεγέθη, αφού ο αριθμός των κλιμάκων είναι περιορισμένος οπότε καλύπτονται λίγες κλίμακες. Με την αύξηση του N , οι κύκλοι αποκτούν μεγαλύτερη ποικιλία σε μέγεθος, με τους μικρότερους να υποδηλώνουν μικρές γωνίες και λεπτομέρειες, ενώ οι μεγαλύτεροι πιο εκτεταμένες δομές. Η δεύτερη παρατήρηση είναι ότι για πολύ μεγάλο N , οι υψηλότερες κλίμακες γίνονται τόσο μεγάλες που ο αλγόριθμος δεν ανιχνεύει πλέον γωνίες, αφού η εικόνα έχει εξομαλυνθεί τόσο πολύ που πλέον αναγνωρίζει ολόκληρες δομές ως γωνίες.

Για διαφορετικές τιμές του s , θα λάβουμε:



Ο παράγοντας κλιμάκωσης s καθορίζει πόσο απέχουν οι διαδοχικές κλίμακες μεταξύ τους. Για μικρό s , οι κλίμακες είναι πυκνά τοποθετημένες με αποτέλεσμα οι κύκλοι να έχουν παρόμοια μεγέθη όπως φαίνεται και παραπάνω. Για μεγάλο s , οι κλίμακες απέχουν περισσότερο, οπότε από την μία εμφανίζεται μεγαλύτερη ποικιλία στα μεγέθη των κύκλων, αλλά από την άλλη, γωνίες των οποίων η χαρακτηριστική κλίμακα πέφτει ανάμεσα σε δύο διαδοχικές κλίμακες του αλγορίθμου, πολύ πιθανόν να μην ανιχνεύονται.

2.3. Ανίχνευση Blobs

Στις προηγούμενες ενότητες, ασχοληθήκαμε με την ανίχνευση γωνιών, δηλαδή σημείων όπου η φωτεινότητα αλλάζει απότομα προς όλες τις κατευθύνσεις. Ωστόσο, σε πολλές εικόνες, υπάρχουν και άλλα χαρακτηριστικά που μας ενδιαφέρουν, όπως οι κηλίδες (blobs).

Γιατί να ανιχνεύσουμε τα blobs;

Η ανίχνευση blobs πρακτικά είναι συμπληρωματική της ανίχνευσης γωνιών, προσφέροντας μια πιο ολοκληρωμένη περιγραφή της εικόνας. Ενώ οι γωνίες μας δίνουν πληροφορία για σημεία ασυνέχειας, τα blobs μας δίνουν πληροφορία για περιοχές με έντονη τοπική δομή. Είναι ιδιαίτερα χρήσιμα σε εφαρμογές όπως:

- Εντοπισμός αντικειμένων: Ανίχνευση στρογγυλών αντικειμένων
- Ανάλυση υφής: Αναγνώριση περιοχών με συγκεκριμένα μοτίβα
- Ιατρική απεικόνιση: Εντοπισμός όγκων ή κυττάρων

Για την ανίχνευση blobs χρησιμοποιούμε την οριζούσα του πίνακα Hessian. Ο πίνακας Hessian περιέχει τις δεύτερες παραγώγους της εικόνας I_σ :

$$H(x, y) = \begin{bmatrix} L_{xx} & L_{xy} \\ L_{xy} & L_{yy} \end{bmatrix}$$

όπου L_{xx}, L_{xy}, L_{yy} οι δεύτερες παράγωγοι της εξομαλυμένης εικόνας.

Η ορίζουσα του Hessian υπολογίζεται ως:

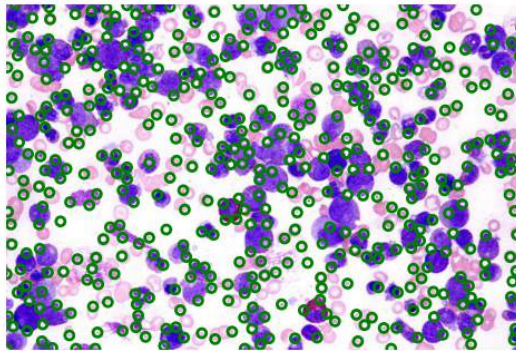
$$\det H = L_{xx} \cdot L_{yy} - L_{xy}^2$$

Το παραπάνω μέγεθος πρακτικά μετρά πόσο έντονα μεταβάλλεται η ένταση και στις δύο κατευθύνσεις ταυτόχρονα. Παίρνει μεγάλες τιμές όταν η φωτεινότητα αλλάζει απότομα προς όλες τις κατευθύνσεις (blob), μικρές τιμές όταν η καμπυλότητα υπάρχει μόνο σε μία κατεύθυνση (ακμή), και μηδενικές όταν δεν υπάρχει καμπυλότητα πουθενά (ομοιόμορφη περιοχή).

Έτσι, εντοπίζοντας τα τοπικά μέγιστα (ή ελάχιστα) της ορίζουσας του Hessian, μπορούμε να ανιχνεύσουμε τις θέσεις και τις κλίμακες όπου υπάρχουν blobs στην εικόνα.

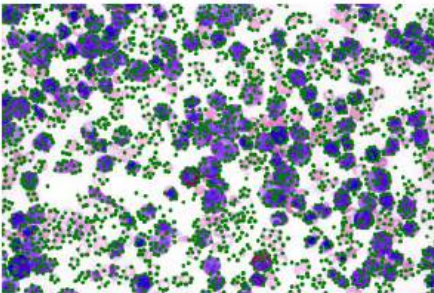
Για την εφαρμογή μας χρησιμοποιήσαμε τις παραμέτρους $\sigma = 2$ και $\theta_{blob} = 0.005$, με θ_{blob} το κατώφλι για την επιλογή των blobs, δηλαδή κρατάμε μόνο τα σημεία όπου η ορίζουσα του Hessian είναι μεγαλύτερη από ένα ποσοστό του ολικού μέγιστου της, αντίστοιχα με το ερώτημα 2.1.3.

blood_cells

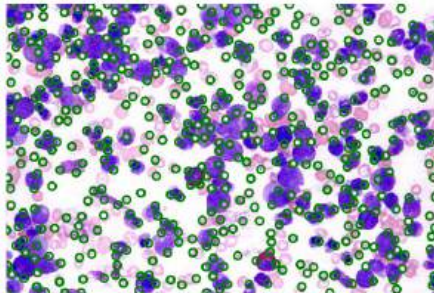


Τώρα, αν αλλάζουμε τις παραμέτρους σ και θ_{blob} , θα λάβουμε τα εξής αποτελέσματα:

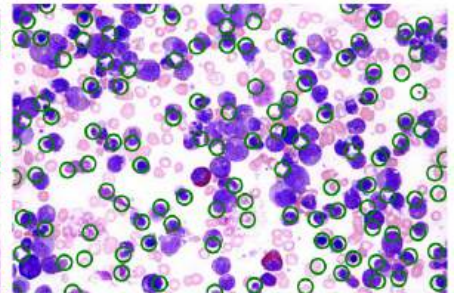
$\sigma = 0.5$



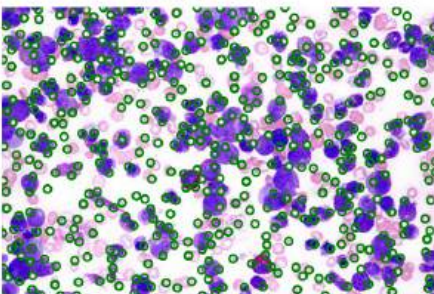
$\sigma = 2$



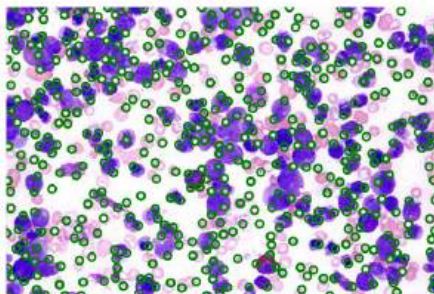
$\sigma = 4.0$



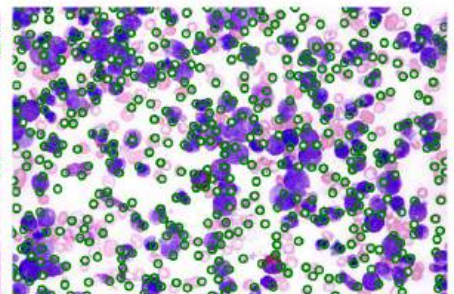
$\theta_{blob} = 0.001$



$\theta_{blob} = 0.005$



$\theta_{blob} = 0.01$



Παρατηρούμε ότι η επίδραση των παραμέτρων είναι ανάλογη με αυτή του ανιχνευτή Harris. Μεγαλύτερο σ ανιχνεύει μεγαλύτερες δομές και μεγαλύτερο θ_{blob} μειώνει τον αριθμό των ανιχνεύσεων.

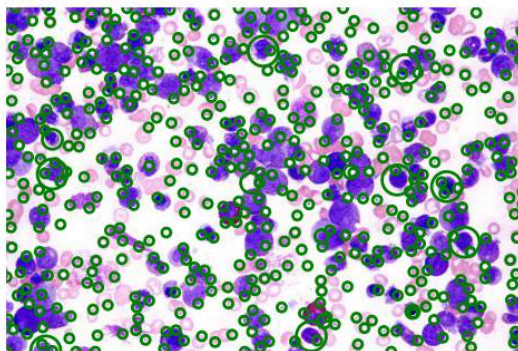
Αν και η λογική επιλογής παραμέτρων είναι παρόμοια, τα ανιχνευμένα σημεία διαφέρουν ποιοτικά. Ο ανιχνευτής **Harris** εντοπίζει σημεία όπου η φωτεινότητα αλλάζει έντονα προς όλες τις κατευθύνσεις ταυτόχρονα, ενώ ο ανιχνευτής **Hessian** εντοπίζει περιοχές με έντονη καμπυλότητα, δηλαδή περιοχές που διαφέρουν σημαντικά από το περιβάλλον τους. Τα δύο είδη σημείων είναι συμπληρωματικά και μαζί δίνουν πιο πλήρη περιγραφή της εικόνας.

2.4. Πολυκλιμακωτή Ανίχνευση Blobs

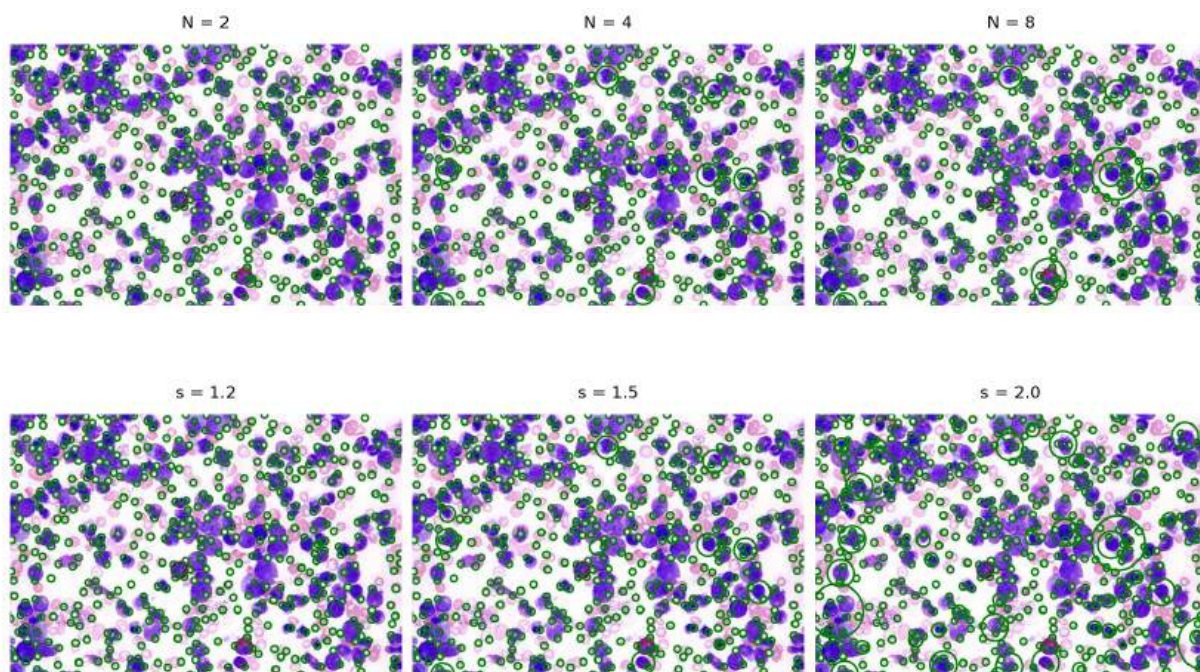
Η διαδικασία πολυκλιμακωτής ανίχνευσης ακολουθεί την ίδια λογική με αυτή της ενότητας 2.2, με τη διαφορά ότι ως κριτήριο ανίχνευσης σε κάθε κλίμακα χρησιμοποιείται η **ορίζουσα του Hessian αντί του Harris**. Η επιλογή της χαρακτηριστικής κλίμακας, γίνεται με τον ίδιο τρόπο με πριν, δηλαδή συγκρίνοντας την τιμή της ορίζουσας με αυτές των γειτονικών κλιμάκων.

Οι παράμετροι που χρησιμοποιήθηκαν για την πολυκλιμακωτή ανίχνευση blobs ήταν ακριβώς οι ίδιοι με αυτούς της ενότητας 2.2 και τα αποτελέσματα που λαμβάνουμε είναι τα εξής:

blood_cells



Αν τώρα αλλάξουμε τις παραμέτρους N και s , θα λάβουμε τα εξής αποτελέσματα, τα οποία είναι ανάλογα αυτών της πολυκλιμακωτής ανίχνευσης γωνιών:



Μέρος 3: Κατηγοριοποίηση Εικόνων με Χρήση Τοπικών Περιγραφητών και Χαρακτηριστικών από Βαθύ Νευρωνικό Δίκτυο

Στο 3^ο και τελευταίο μέρος του Project, θα ασχοληθούμε με την κατηγοριοποίηση εικόνων, αξιοποιώντας στο πρώτο μισό τα σημεία ενδιαφέροντος και τις κλίμακες που εξάγαμε στο 2^ο μέρος. Πιο συγκεκριμένα, χρησιμοποιώντας εικόνες τριών κλάσεων (αυτοκίνητο, άνθρωπος, ποδήλατο) από τη βάση Pascal VOC2005, στόχος μας είναι η ταξινόμηση κάθε εικόνας στη σωστή κλάση.

Για την επίτευξη αυτού, θα ακολουθήσουμε δύο διαφορετικές προσεγγίσεις. Η πρώτη βασίζεται σε κλασικούς τοπικούς περιγραφητές (SURF, HOG) και την αναπαράσταση Bag of Visual Words (BoVW), όπου τα τοπικά χαρακτηριστικά κωδικοποιούνται σε ένα συνολικό ιστόγραμμα για κάθε εικόνα. Η δεύτερη, αξιοποιεί ένα προεκπαιδευμένο συνελικτικό νευρωνικό δίκτυο (MobileNet V3) για εξαγωγή χαρακτηριστικών μέσω Transfer Learning, κρατώντας τις ενεργοποιήσεις του δικτύου πριν το τελικό επίπεδο ταξινόμησης.

Και στις δύο περιπτώσεις, η τελική κατηγοριοποίηση θα πραγματοποιηθεί με ταξινομητή SVM, επιτρέποντας έτσι την άμεση σύγκριση των δύο προσεγγίσεων ως προς την απόδοσή τους. Επιπλέον, θα μελετήσουμε την ευρωστία των μεθόδων σε παραμορφώσεις όπως η προσθήκη θορύβου και η περιστροφή, εξάγοντας συμπεράσματα για τα πλεονεκτήματα και τις αδυναμίες κάθε προσέγγισης.

Σημείωση: Το Bag of Visual Words (BoVW) είναι μέθοδος αναπαράστασης εικόνων που μετατρέπει τους τοπικούς περιγραφητές σε ένα σταθερής διάστασης διάνυσμα. Αρχικά, δημιουργείται ένα οπτικό λεξικό με k-means πάνω στους περιγραφητές του συνόλου εκπαίδευσης. Στη συνέχεια, κάθε εικόνα αναπαρίσταται ως ιστόγραμμα συχνοτήτων εμφάνισης των οπτικών λέξεων, κανονικοποιημένο με την L2 νόρμα του.

3.1. Κατηγοριοποίηση με τοπικούς περιγραφητές

Τι είναι οι τοπικοί περιγραφητές;

Οι τοπικοί περιγραφητές είναι διανύσματα αριθμών που κωδικοποιούν την πληροφορία μιας μικρής γειτονιάς γύρω από ένα σημείο ενδιαφέροντος. Αντί να κρατάμε όλα τα pixel της περιοχής, τα συνοψίζουμε σε ένα συμπαγές διάνυσμα το οποίο περιγράφει τα χαρακτηριστικά της περιοχής, όπως οι κλίσεις, η υφή και τα μοτίβα.

Γιατί τους χρειαζόμαστε;

- **Συμπαγής αναπαράσταση:** Μειώνουν δραστικά τη διάσταση των δεδομένων (π.χ. από 120.000 pixels σε 2.560 αριθμούς)
- **Επαναληψιμότητα:** Οι περιγραφητές δίνουν παρόμοιες τιμές για την ίδια φυσική περιοχή ακόμα κι αν η εικόνα έχει υποστεί μετασχηματισμούς (περιστροφή, αλλαγή φωτισμού, ...)
- **Κατηγοριοποίηση:** Επιτρέπουν την κωδικοποίηση κάθε εικόνας σε ένα συνολικό διάνυσμα χαρακτηριστικών μέσω της αναπαράστασης Bag of Visual Words, το οποίο στη συνέχεια τροφοδοτείται στον ταξινομητή

Πώς δουλεύουν γενικά;

Οι περισσότεροι τοπικοί περιγραφητές ακολουθούν την εξής λογική:

1. **Επιλογή γειτονιάς:** Γύρω από κάθε σημείο ενδιαφέροντος ορίζεται μια περιοχή με μέγεθος ανάλογο της κλίμακας του σημείου
2. **Υποδιαίρεση:** Η γειτονιά χωρίζεται σε μικρότερα υποτιμήματα
3. **Υπολογισμός χαρακτηριστικών:** Σε κάθε υποτιμήμα καταγράφεται η κατανομή των κλίσεων (π.χ. πόσες ακμές είναι οριζόντιες, κάθετες ή διαγώνιες)
4. **Κωδικοποίηση:** Όλες οι πληροφορίες ενώνονται σε ένα ενιαίο διάνυσμα

Οι δύο τοπικοί περιγραφητές που θα χρησιμοποιήσουμε είναι οι εξής:

- **SURF:** Ο περιγραφητής SURF (Speed Up Robust Features) βασίζεται στις **κλίσεις** της εικόνας μέσα στη γειτονιά του σημείου ενδιαφέροντος. Αρχικά, υπολογίζει τη **γενική κατεύθυνση της γειτονιάς** χρησιμοποιώντας ιστογράμματα κλίσεων και στη συνέχεια περιστρέφει τη γειτονιά κατάλληλα ώστε η κατεύθυνση της να ευθυγραμμιστεί με έναν σταθερό άξονα. Μόνο τότε υπολογίζεται ο περιγραφητής. Έτσι, αυτός γίνεται **περιστροφικά αναλλοίωτος**, δηλαδή η ίδια περιοχή δίνει τον ίδιο περιγραφητή ανεξάρτητα από τη γωνία λήψης της εικόνας. Στη συνέχεια, η γειτονιά χωρίζεται σε 4×4 υποπεριοχές και σε κάθε μια υπολογίζονται στατιστικά στοιχεία αναφορικά με τις κλίσεις (άθροισμα αποκρίσεων Haar wavelet και απόλυτων τιμών των αποκρίσεων σε οριζόντιες και κατακόρυφες κατευθύνσεις), παράγοντας το τελικό διάνυσμα 64 διαστάσεων.

- **HOG:** Ο περιγραφητής HOG (Histogram of Oriented Gradients) περιγράφει τη γειτονιά μέσω της κατανομής των κατευθύνσεων των κλίσεων. Η διαδικασία υπολογισμού του ακολουθεί τα εξής βήματα:
 - **Υπολογισμός κλίσεων:** Για κάθε pixel της γειτονιάς υπολογίζονται η κατεύθυνση και το μέγεθος της κλίσης
 - **Χωρισμός σε κελιά:** Η γειτονιά χωρίζεται σε μικρά κελιά (π.χ. 8×8 pixels)
 - **Δημιουργία ιστογραμμάτων:** Σε κάθε κελί κατασκευάζεται ένα ιστόγραμμα με 9 ή 12 bins (κατευθύνσεις π.χ. $0^\circ - 20^\circ, 20^\circ - 40^\circ, \dots$), όπου κάθε pixel συνεισφέρει στο αντίστοιχο bin με βάρος ίσο με το μέγεθος της κλίσης του
 - **Ομαδοποίηση και κανονικοποίηση:** Τα κελιά ομαδοποιούνται σε μεγαλύτερα μπλοκ (π.χ. 2×2 κελιά), των οποίων κανονικοποιούμε τα ιστογράμματα ώστε να είναι πιο ανθεκτικά σε αλλαγές φωτεινότητας
 - **Σχηματισμός διανύσματος:** Όλα τα κανονικοποιημένα ιστογράμματα ενώνονται σε ένα ενιαίο διάνυσμα, του οποίου η διάσταση εξαρτάται από το μέγεθος της γειτονιάς, το πλήθος των κελιών και τον αριθμό των bins

Σε αντίθεση με τον SURF, ο HOG δεν υπολογίζει γενική κατεύθυνση, άρα δεν είναι περιστροφικά αναλλοίωτος. Είναι όμως ιδιαίτερα αποτελεσματικός στην αναγνώριση σχημάτων με σταθερό προσανατολισμό, όπως ανθρώπινες φιγούρες, ζώα και οχήματα αφού η λεπτομερής κατανομή των κλίσεων ανά κελί αποτυπώνει με ακρίβεια τη μορφή του αντικειμένου, κάτι που ο ταξινομητής μπορεί να εκμεταλλευτεί άμεσα.

Για την κατηγοριοποίηση των εικόνων ακολουθήσαμε την εξής διαδικασία. Αρχικά, για κάθε εικόνα εξήχθησαν τοπικοί περιγραφητές γύρω από τα σημεία ενδιαφέροντος που ανιχνεύθηκαν με τους πολυκλιμακωτούς ανιχνευτές που φτιάξαμε στο 2^ο μέρος. Στη συνέχεια, πειραματιστήκαμε με τους τέσσερις συνδυασμούς ανιχνευτή/περιγραφητή: multi-scale corners με SURF και HOG, και multi-scale blobs με SURF και HOG. Έστερα, κωδικοποιήσαμε τα χαρακτηριστικά κάθε εικόνας με την αναπαράσταση Bag of Visual Words. Συγκεκριμένα, δημιουργήθηκε ένα οπτικό λεξικό 500 λέξεων (clusters). Τέλος, η κατηγοριοποίηση πραγματοποιήθηκε με ταξινομητή SVM πολλαπλών κλάσεων. Για την αξιολόγηση χρησιμοποιήθηκε 5-fold cross-validation.

Τα αποτελέσματα παρουσιάζονται στον παρακάτω πίνακα:

Ανιχνευτής	Περιγραφητής	Mean Accuracy
Multi-scale corners	SURF	74.207%
Multi-scale corners	HOG	64%
Multi-scale blobs	SURF	73.793%
Multi-scale blobs	HOG	62.897%

Table 1: Αποτελέσματα κατηγοριοποίησης με τοπικούς περιγραφητές

Από τα αποτελέσματα του παραπάνω πίνακα προκύπτουν τα εξής συμπεράσματα:

1. Ο περιγραφητής **SURF δίνει καλύτερα αποτελέσματα έναντι του HOG και για τους δύο ανιχνευτές**, με διαφορά περίπου 10 – 11%. Αυτό αποδίδεται στο γεγονός ότι ο SURF είναι περιστροφικά αναλλοίωτος όπως αναφέραμε και προηγουμένως, με αποτέλεσμα να παράγει πιο αξιόπιστους περιγραφητές για εικόνες που εμφανίζουν ποικιλία προσανατολισμών. Ο HOG από την άλλη, αν και αποτελεσματικός σε προβλήματα με σταθερό προσανατολισμό, φαίνεται να υστερεί λόγω της ποικιλομορφίας των εικόνων.
2. Ως προς τους ανιχνευτές, **ο multi-scale corners δίνει ελαφρώς καλύτερα αποτελέσματα από τον multi-scale blobs** και για τους δύο περιγραφητές. Η διαφορά αυτή είναι μικρή και υποδηλώνει ότι και οι δύο ανιχνευτές εντοπίζουν παρόμοια χαρακτηριστικά σημεία, με τις γωνίες να παρέχουν ελαφρώς πιο διακριτική πληροφορία για το πρόβλημα της κατηγοριοποίησης.
3. Συνολικά, ο συνδυασμός **multi-scale corners + SURF** με accuracy 74.207% αποτελεί την **καλύτερη επιλογή** από τους κλασικούς τοπικούς περιγραφητές για το συγκεκριμένο πρόβλημα κατηγοριοποίησης.

3.2. Κατηγοριοποίηση με χαρακτηριστικά από συνελκτικό νευρωνικό δίκτυο

Αφού είδαμε πως μπορεί να γίνει η ταξινόμηση με τη μέθοδο των τοπικών περιγραφών, τώρα μπορούμε να περάσουμε σε έναν εναλλακτικό τρόπο υλοποίησης, δηλαδή με συνελκτικά νευρωνικά δίκτυα.

Τα Συνελκτικά Νευρωνικά Δίκτυα (CNNs), κυριαρχούν τα τελευταία χρόνια στα προβλήματα αναγνώρισης αντικειμένων. Η αρχιτεκτονική τους αποτελείται από 3 βασικά επίπεδα:

- **Συνελκτικά Επίπεδα (convolutional layers):** Εφαρμόζουν φίλτρα με σκοπό την ανίχνευση τοπικών χαρακτηριστικών
- **Επίπεδα Υποδειγματοληψίας (pooling layers):** Επίπεδα που μειώνουν τη χωρική διάσταση διατηρώντας τα πιο σημαντικά χαρακτηριστικά ανά περιοχή
- **Πλήρως Συνδεδεμένα Επίπεδα (fully connected layers):** Επίπεδα που συνδυάζουν τα χαρακτηριστικά ώστε στη συνέχεια να γίνει η τελική ταξινόμηση

Πρακτικά, τα αρχικά επίπεδα είναι υπεύθυνα για την ανίχνευση απλών χαρακτηριστικών όπως ακμές και γωνίες, ενώ τα βαθύτερα επίπεδα κοιτάνε για πιο σύνθετες αναπαραστάσεις και δομές.

Ένα από τα βασικά αρνητικά των CNNs είναι ότι απαιτούν πάρα πολλά δεδομένα και υπολογιστικούς πόρους. Για αυτό, αντί να εκπαιδεύουμε κάθε φορά ένα νέο δίκτυο από την αρχή, μέσω της μεθόδου του **Transfer Learning**, χρησιμοποιούμε προεκπαιδευμένα μοντέλα, τα οποία γνωρίζουν πως να διαχωρίζουν χαρακτηριστικά. Έτσι, αφαιρούμε το τελευταίο επίπεδο ταξινόμησης (classifier head) και χρησιμοποιούμε την έξοδο ενός ενδιάμεσου επιπέδου ως διάνυσμα χαρακτηριστικών (feature vector). Αυτό το διάνυσμα αποτελεί μια γενική αναπαράσταση της εικόνας, η οποία μπορεί στη συνέχεια να δοθεί ως είσοδος σε έναν κλασικό ταξινομητή, όπως ο SVM.

Αν κρατούσαμε το classifier head, το δίκτυο θα παρήγαγε πιθανότητες και για τις 1000 κλάσεις του ImageNet, οι οποίες δεν σχετίζονται με το δικό μας πρόβλημα των τριών κλάσεων. Για το λόγο αυτό, αξιοποιούμε μόνο τα χαρακτηριστικά που έχει μάθει το δίκτυο και εκπαιδεύουμε έναν νέο ταξινομητή προσαρμοσμένο στο πρόβλημα μας.

Για τις ανάγκες του συγκεκριμένου ζητήματος, θα κάνουμε χρήση του **MobileNet V3 Small**, ενός υπολογιστικά ελαφρού CNN, το οποίο αποτελείται από 15 επίπεδα και χρησιμοποιεί **depthwise seperable convolutions** με σκοπό τη μείωση του αριθμού των παραμέτρων σε σχέση με τα κλασικά CNNs.

Σημείωση: Οι depthwise separable convolutions είναι μια υπολογιστικά αποδοτική εναλλακτική της κλασικής συνέλιξης. Αντί να εφαρμόζεται ένα φίλτρο ταυτόχρονα σε όλα τα κανάλια, η πράξη χωρίζεται σε δύο βήματα: πρώτα μια depthwise convolution που εφαρμόζει ένα ξεχωριστό φίλτρο σε κάθε κανάλι independently, και μετά μια pointwise convolution (1×1) που συνδυάζει τα αποτελέσματα. Αυτό μειώνει δραστικά τον αριθμό των παραμέτρων και των υπολογισμών, καθιστώντας το μοντέλο κατάλληλο για περιβάλλοντα με περιορισμένους πόρους.

3.2.1. Φόρτωση Μοντέλου και Προεπεξεργασία Εικόνων

Τα νευρωνικά δίκτυα απαιτούν η είσοδος να έχει ακριβώς την ίδια μορφή με τις εικόνες εκπαίδευσης φυσικά. Πιο συγκεκριμένα, το MobileNet V3 Small έχει εκπαιδευτεί σε RGB εικόνες διαστάσεων 224×224 pixels, κανονικοποιημένες με τη μέση τιμή και τυπική απόκλιση του ImageNet. Επίσης, το pytorch δέχεται αποκλειστικά tensors και όχι numpy εικόνες.

Για αυτούς τους λόγους, έπρεπε πριν την εξαγωγή των χαρακτηριστικών, να κάνουμε τους εξής μετασχηματισμούς:

- `img = Image.open(...).convert('RGB')`
Διαβάζει την εικόνα και τη μετατρέπει σε μορφή RGB. Η μετατροπή αυτή εξασφαλίζει ότι όλες οι εικόνες έχουν το ίδιο format εισόδου, ανεξάρτητα από τον αρχικό τους τύπο.
- `img_tensor = preprocess(img)`
Εφαρμόζει μια σειρά από μετασχηματισμούς στην εικόνα, ώστε να έχει την ίδια μορφή με τις εικόνες στις οποίες εκπαιδεύτηκε το προεκπαιδευμένο μοντέλο. Συγκεκριμένα, πραγματοποιεί αλλαγή μεγέθους, κεντρικό cropping, μετατροπή σε tensor και κανονικοποίηση με τη μέση τιμή και την τυπική απόκλιση του ImageNet.
- `img_tensor = img_tensor.unsqueeze(0)`
Προσθέτει μία επιπλέον διάσταση στην είσοδο (batch dimension), μετατρέποντας το tensor από διάσταση (C, H, W) σε $(1, C, H, W)$, όπως απαιτείται από τα νευρωνικά δίκτυα στο PyTorch για επεξεργασία σε batches.

3.2.2. Εξαγωγή Χαρακτηριστικών

Στο δεύτερο βήμα, θέλουμε να εξάγουμε τα χαρακτηριστικά των εικόνων μας μέσω του μοντέλου μας. Για αυτό, αρχικά θέτουμε το μοντέλο σε κατάσταση αξιολόγησης (`model.eval()`), απενεργοποιώντας επίπεδα όπως το Dropout και το BatchNorm που χρησιμοποιούνται μόνο κατά την εκπαίδευση. Επιπλέον, απενεργοποιούμε τον υπολογισμό παραγώγων με `torch.no_grad()`, καθώς σε αυτήν τη φάση δεν εκπαιδεύουμε το δίκτυο και ο υπολογισμός τους θα σπαταλούσε μνήμη χωρίς κανένα όφελος. Χρησιμοποιώντας τη συνάρτηση `create_feature_extractor`, διατηρούμε την έξοδο του επιπέδου `avgpool` και αφαιρούμε το τελικό επίπεδο ταξινόμησης.

Με αυτόν τον τρόπο, για κάθε εικόνα εξάγεται ένα διάνυσμα χαρακτηριστικών διαστάσεων 1280. Συγκεκριμένα, η έξοδος του `avgpool` έχει αρχικά σχήμα $(1, 1280, 1, 1)$, το οποίο, με τη χρήση της συνάρτησης `squeeze()`, μετατρέπεται σε μονοδιάστατο διάνυσμα χαρακτηριστικών διαστάσεων $(1280,)$.

Σημείωση: Η έξοδος του `avgpool` layer έχει αρχικά σχήμα $(1, 1280, 1, 1)$, όπου το 1280 αντιστοιχεί στον αριθμό των εξαγόμενων χαρακτηριστικών και οι υπόλοιπες διαστάσεις αντιστοιχούν στο batch size και στις χωρικές διαστάσεις. Με τη χρήση της συνάρτησης `squeeze()`, αφαιρούνται οι διαστάσεις που είναι ίσες με μονάδα, με αποτέλεσμα την παραγωγή ενός μονοδιάστατου διανύσματος χαρακτηριστικών μήκους 1280.

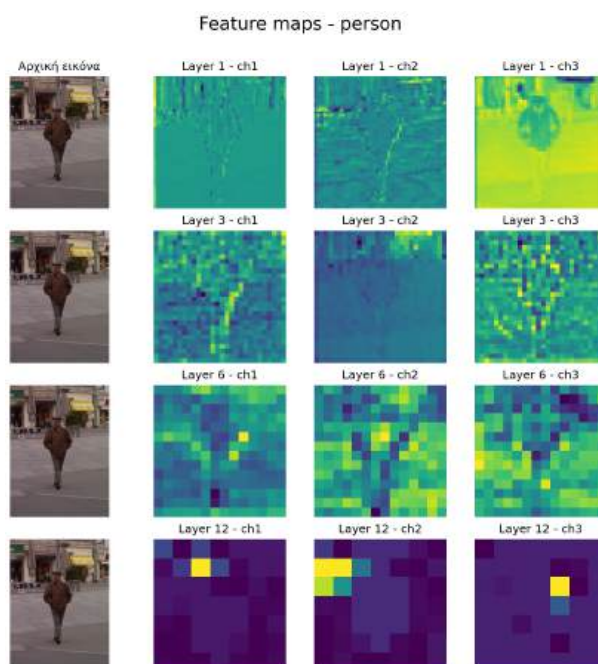
3.2.3. Δημιουργία Συνόλων Train, Test και Αξιολόγηση

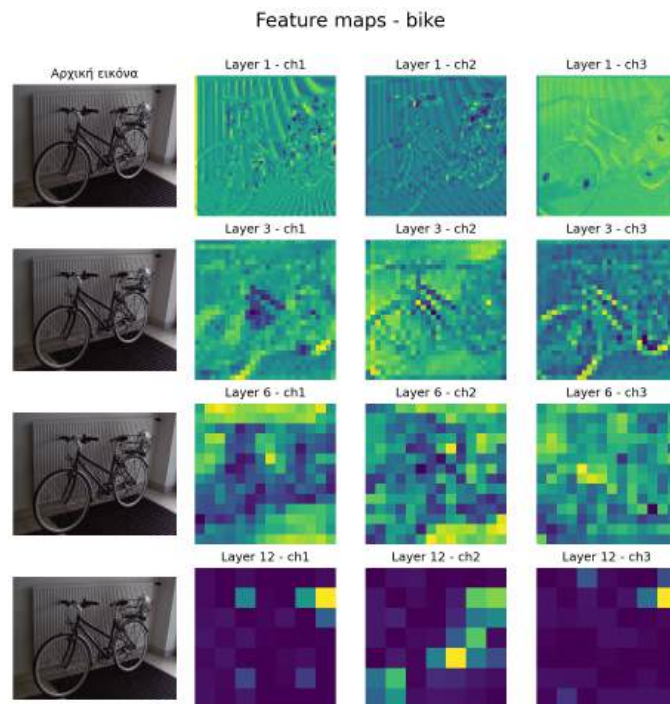
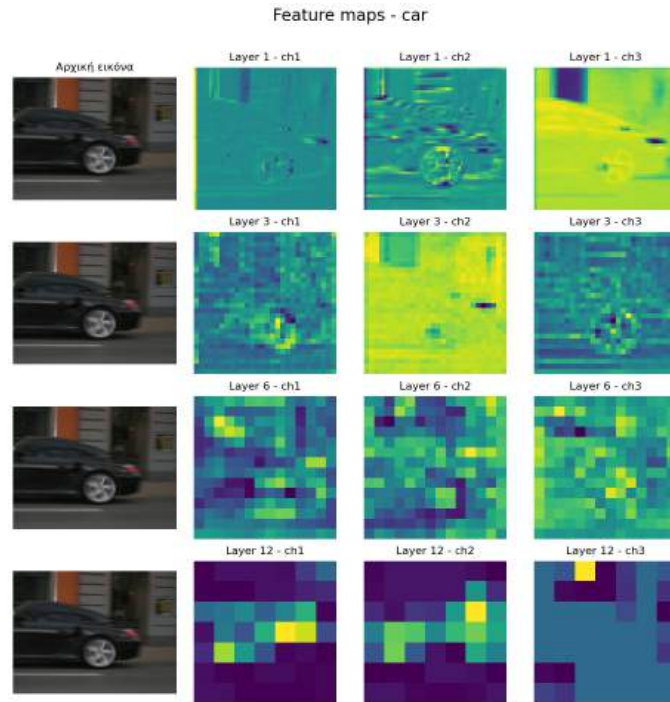
Όπως και στο ερώτημα 3.1.2, χωρίζουμε αρχικά τις εικόνες μας σε σύνολα εκπαίδευσης και δοκιμής. Στη συνέχεια, αντί να χρησιμοποιήσουμε την αναπαράσταση Bag of Visual Words, εκπαιδεύουμε το SVM απευθείας πάνω στα χαρακτηριστικά του MobileNet. Αυτό είναι εφικτό επειδή το MobileNet παράγει ήδη **για κάθε εικόνα ένα διάνυσμα σταθερού μεγέθους** (1280 χαρακτηριστικά). Έτσι, δεν χρειάζεται η χρήση Bag of Visual Words, όπως συμβαίνει με τους τοπικούς περιγραφητές, όπου ο αριθμός των χαρακτηριστικών διαφέρει από εικόνα σε εικόνα. Τα features μετατράπηκαν σε numpy arrays και τροφοδοτήθηκαν απευθείας στο SVM, ακολουθώντας την ίδια διαδικασία 5-fold cross-validation με το ερώτημα 3.1.4.

Τελικά, το MobileNet πέτυχε **mean accuracy** 95.310%, ένα μέγεθος αρκετά υψηλότερο από τους τοπικούς περιγραφητές που είδαμε πριν. Προφανώς, αυτό ήταν αναμενόμενο αφού τα χαρακτηριστικά στη δεύτερη υλοποίηση εξήχθησαν από ένα δίκτυο εκπαιδευμένο σε ένα τεράστιο αριθμό εικόνων, το οποίο, λόγω των πολλών στρωμάτων του, μπορεί να διαχωρίσει εύκολα χαρακτηριστικά χαμηλού και υψηλού επιπέδου όπως αναφέραμε και προηγουμένως.

3.2.4. Feature Maps

Αφού υλοποιήσαμε όλα τα προηγούμενα βήματα, μπορούμε τώρα να οπτικοποιήσουμε τις ενεργοποιήσεις των ενδιάμεσων συνελικτικών επιπέδων του νευρωνικού μας δικτύου. Ενδεικτικά, διαλέξαμε μια φωτογραφία από κάθε κατηγορία, για την οποία παρουσιάζουμε τα πρώτα 3 κανάλια 4^{ων} layers της.





Πιο συγκεκριμένα, **κάθε κανάλι αντιστοιχεί σε ένα διαφορετικό φίλτρο που έχει μάθει να ανιχνεύει διαφορετικά χαρακτηριστικά**. Τα χρώματα στην οπτικοποίηση (colormap viridis) αντιπροσωπεύουν την ένταση ενεργοποίησης. Οι φωτεινές (κίτρινες) περιοχές αντιστοιχούν σε υψηλή ενεργοποίηση, δηλαδή το φίλτρο αναγνώρισε κάτι σημαντικό εκεί, ενώ οι σκούρες (μπλε) περιοχές αντιστοιχούν σε χαμηλή ενεργοποίηση. Στα πρώτα layers (Layer 1, Layer 3), τα feature maps μοιάζουν πολύ με την αρχική εικόνα, καθώς διατηρούν πολλές λεπτομέρειες. Σε αυτά τα επίπεδα, όπως αναφέραμε και προηγουμένως, το δίκτυο εντοπίζει απλά χαρακτηριστικά όπως ακμές, γωνίες και αλλαγές φωτεινότητας, δηλαδή βασικά στοιχεία της εικόνας. Καθώς προχωράμε σε βαθύτερα layers (Layer 6, Layer 12), τα feature maps γίνονται όλο και λιγότερο αναγνωρίσιμα. Το δίκτυο έχει πλέον συνδυάσει τα αρχικά χαρακτηριστικά και εντοπίζει πιο σύνθετα στοιχεία, όπως υφές ή μέρη αντικειμένων. Παράλληλα, η χωρική ανάλυση (λεπτομέρεια μιας εικόνας στο χώρο) μειώνεται, με αποτέλεσμα να χάνεται η λεπτομέρεια της αρχικής εικόνας (κυρίως λόγω των pooling layers που αφαιρούν πληροφορία). Παρόλο που τα feature maps των βαθύτερων layers δεν μοιάζουν με την αρχική εικόνα, περιέχουν πιο σημαντική πληροφορία για την αναγνώριση του αντικειμένου, γεγονός που εξηγεί την υψηλή ακρίβεια του μοντέλου.

3.2.5 Επίδραση Παραμορφώσεων στην Απόδοση

Στο τελευταίο μέρος της άσκησης, θέλουμε να δούμε πώς παραμορφώσεις όπως η **προσθήκη θορύβου** και η **περιστροφή** επηρεάζουν την απόδοση του μοντέλου μας. Για αυτό, θα επαναλάβουμε τα πειράματα του μέρους 3.1 και 3.2, ώστε να δούμε τι επίδραση έχουν οι παραμορφώσεις στην απόδοση των δύο μεθόδων.

Για τον **Gaussian θόρυβο**, προστέθηκε σε κάθε εικόνα τυχαίος θόρυβος με κατανομή $N(0, 35)$, δηλαδή θόρυβος με μέση τιμή 0 και τυπική απόκλιση 35. Ο θόρυβος που προστέθηκε επηρεάζει ορατά την εικόνα χωρίς όμως να την καταστρέφει τελείως.

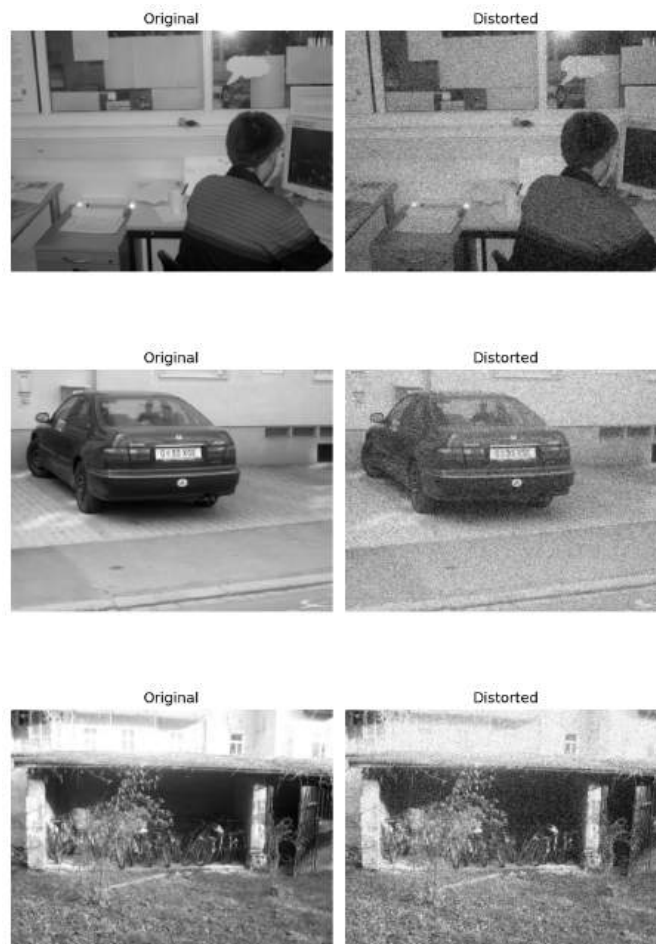
Για την **περιστροφή**, κάθε εικόνα περιστράφηκε κατά τυχαία γωνία ομοιόμορφα κατανομημένη στο διάστημα $[-45^\circ, 45^\circ]$. Γενικά, αναμένουμε ο **SURF** να είναι πιο ανθεκτικός στην περιστροφή από τον **HOG**, καθώς υπολογίζει τη γενική κατεύθυνση της γειτονιάς πριν την κωδικοποίηση όπως περιγράψαμε και προηγουμένως, καθιστώντας τον περιστροφικά αναλλοίωτο.

Αντίθετα, το **MobileNet** αναμένεται να παρουσιάζει μεγαλύτερη ευρωστία και στις δύο μορφές παραμόρφωσης, καθώς έχει εκπαιδευτεί σε εκτεταμένο και ποικιλόμορφο σύνολο δεδομένων, το οποίο περιλαμβάνει εικόνες με διαφορετικά επίπεδα θορύβου και περιστροφές.

Σημείωση: Για την υλοποίηση του ερωτήματος, χρησιμοποιήθηκε η μεταβλητή **distort**, η οποία ήταν ήδη ορισμένη στη συνάρτηση *FeatureExtraction* του δοσμένου υλικού.

3.1 με Gaussian Θόρυβο

Κάποια παραδείγματα εικόνων μετά την προσθήκη θορύβου είναι:



Κάνοντας χρήση των ίδιων σταθερών όπως και στο ερώτημα 3.1, τα αποτελέσματα που λάβαμε ήταν τα εξής:

Ανιχνευτής	Περιγραφητής	Mean Accuracy
Multi-scale corners	SURF	72.414%
Multi-scale corners	HOG	56.276%
Multi-scale blobs	SURF	71.448%
Multi-scale blobs	HOG	57.241%

Table 2: Αποτελέσματα κατηγοριοποίησης με τοπικούς περιγραφητές (με θορυβώδη δεδομένα)

Όπως αναμενόταν, η προσθήκη θορύβου **επηρέασε αρνητικά την απόδοση** των τοπικών περιγραφητών, με τη mean accuracy να μειώνεται κατά περίπου 2 – 8% σε σχέση με τα αποτελέσματα χωρίς παραμόρφωση.

Ωστόσο, παρατηρείται ότι ο **HOG παρουσιάζει μεγαλύτερη πτώση** (περίπου 6 – 8%) σε σύγκριση με τον SURF (περίπου 2 – 3%). Το αποτέλεσμα αυτό είναι αναμενόμενο, καθώς ο SURF βασίζεται σε υπολογισμούς που είναι πιο ανθεκτικοί σε τοπικές μεταβολές φωτεινότητας και μικρές διακυμάνσεις, ενώ ο HOG εξαρτάται από ιστογράμματα κατευθύνσεων, τα οποία επηρεάζονται πιο έντονα από τυχαίες μεταβολές στις τιμές των pixels.

Τέλος, αξίζει να σημειωθεί ότι οι παρατηρούμενες διαφορές, ειδικά στον SURF, είναι συγκρίσιμες με τη φυσική διακύμανση των αποτελεσμάτων (περίπου 2 – 3%), η οποία οφείλεται στην τυχαία φύση της διαδικασίας 5-fold cross-validation.

3.1 με Περιστροφή

Κάποια παραδείγματα εικόνων μετά την περιστροφή είναι:



Κάνοντας χρήση των ίδιων σταθερών όπως και στο ερώτημα 3.1, τα αποτελέσματα που λάβαμε ήταν τα εξής:

Ανιχνευτής	Περιγραφητής	Mean Accuracy
Multi-scale corners	SURF	68.690%
Multi-scale corners	HOG	52.966%
Multi-scale blobs	SURF	67.586%
Multi-scale blobs	HOG	54.207%

Table 3: Αποτελέσματα κατηγοριοποίησης με τοπικούς περιγραφητές (με περιστροφή)

Με την περιστροφή των εικόνων, η απόδοση μειώνεται περίπου κατά 5–10% για όλους τους συνδυασμούς.

Ο HOG επηρεάζεται περισσότερο, με τον συνδυασμό corners + HOG να πέφτει κάτω από το 53%.

Αντίθετα, ο SURF φαίνεται να αντέχει καλύτερα, καθώς υπολογίζει πρώτα τη γενική κατεύθυνση της περιοχής πριν δημιουργήσει την περιγραφή, κάτι που τον κάνει θεωρητικά ανεξάρτητο από την περιστροφή. Παρ' όλα αυτά, παρατηρείται και εκεί μια πτώση στην απόδοση, κάτι που δείχνει ότι αυτή η ιδιότητα δεν ισχύει πλήρως στην πράξη, ειδικά όταν χρησιμοποιείται μαζί με το BoVW που έχει εκπαιδευτεί σε εικόνες χωρίς περιστροφή.

Συνολικά, τα αποτελέσματα μας δείχνουν ότι η **περιστροφή είναι πιο καταστροφική από τον θόρυβο** για δύο λόγους:

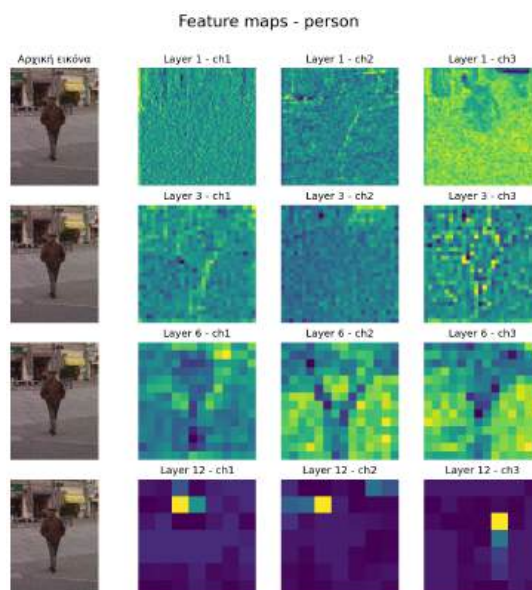
- Πρώτον, **αλλάζει δομικά την εικόνα**, τα σημεία ενδιαφέροντος που ανιχνεύονται σε μια περιστραμμένη εικόνα μπορεί να είναι εντελώς διαφορετικά από αυτά της αρχικής, οπότε οι περιγραφητές που εξάγονται δεν αντιστοιχούν στις ίδιες φυσικές περιοχές.
- Δεύτερον, το **οπτικό λεξικό (BoVW)** έχει κατασκευαστεί από μη-περιστραμμένες εικόνες, οπότε οι οπτικές λέξεις που αναθέτει σε περιστραμμένες εικόνες μπορεί να είναι λανθασμένες, επιδεινώνοντας περαιτέρω την απόδοση.

3.2 με Θόρυβο

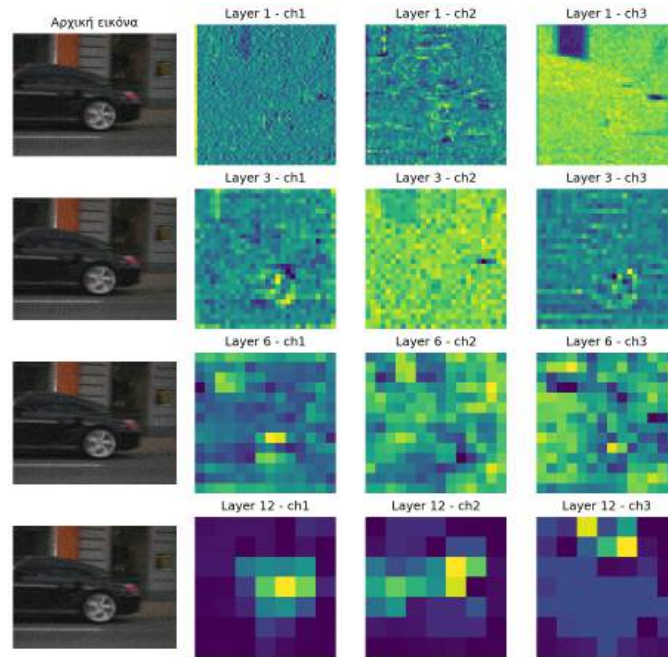
Για το MobileNet με προσθήκη Gaussian θορύβου, η απόδοση μειώθηκε στο 93.241%, έναντι του 95.310% χωρίς παραμόρφωση.

Δηλαδή, μια πτώση μόλις 2.1%. Αυτό επιβεβαιώνει την **ανθεκτικότητα του MobileNet στον θόρυβο**, καθώς τα βαθιά χαρακτηριστικά που εξάγονται από το δίκτυο είναι πολύ πιο εύρωστα από τους τοπικούς περιγραφητές. Το δίκτυο, όπως αναφέραμε και προηγουμένως, έχει εκπαιδευτεί σε τεράστιο σύνολο δεδομένων με ποικιλία συνθηκών, αναπτύσσοντας αναπαραστάσεις που δεν εξαρτώνται από τοπικές διακυμάνσεις φωτεινότητας.

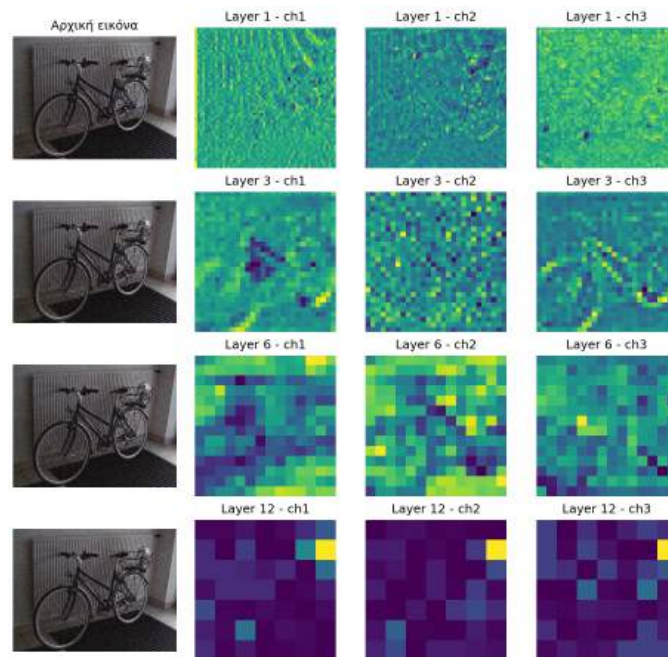
Στη συνέχεια, εξετάζουμε την επίδραση της προσθήκης Gaussian θορύβου στα feature maps του δικτύου. Στα παρακάτω σχήματα παρουσιάζονται οι αντίστοιχες ενεργοποιήσεις για τις ίδιες εικόνες, μετά την εφαρμογή θορύβου:



Feature maps - car



Feature maps - bike



Παρατηρούμε ότι ο θόρυβος της εισόδου δεν απορρίπτεται πλήρως από το δίκτυο, αλλά **μεταδίδεται στα feature maps**, επηρεάζοντας τις ενεργοποιήσεις των φίλτρων. Σε κάποιες περιπτώσεις, ο θόρυβος φαίνεται ενισχυμένος, αφού τα συνελκτικά φίλτρα ανιχνεύουν και υψηλές συχνότητες, στις οποίες συμπεριλαμβάνονται και οι τυχαίες μεταβολές του Gaussian θορύβου.

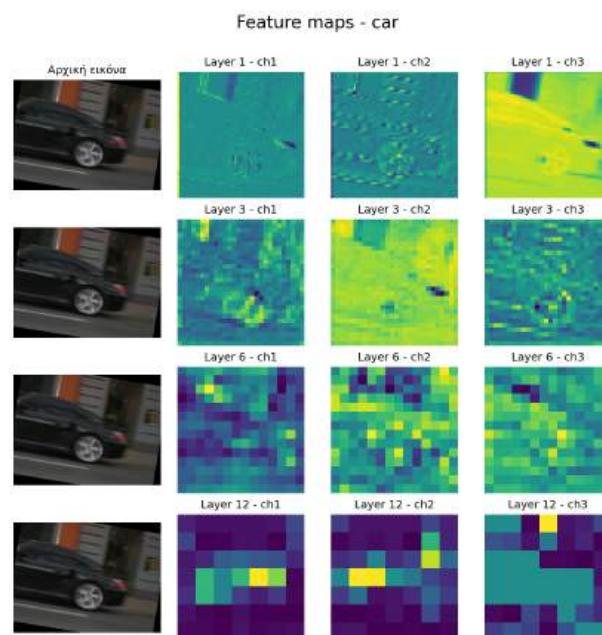
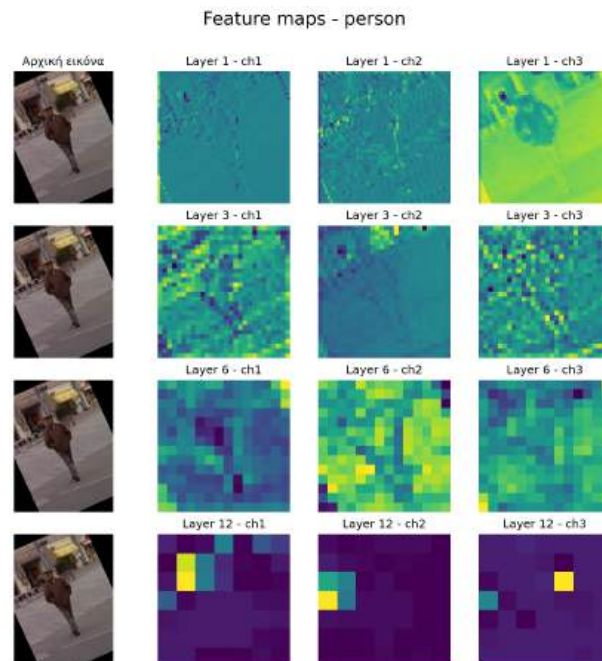
Ιδιαίτερα σημαντική είναι η παρατήρηση ότι η επίδραση του θορύβου είναι **εντονότερη στα αρχικά (χαμηλότερα) layers** του δικτύου. Αυτό είναι αναμενόμενο, καθώς όπως αναφέραμε και πριν, τα πρώτα layers λειτουργούν ως ανιχνευτές απλών χαρακτηριστικών, όπως ακμές και τοπικές μεταβολές φωτεινότητας, τα οποία επηρεάζονται άμεσα από τον θόρυβο. Αντίθετα, στα βαθύτερα layers παρατηρούμε ότι οι ενεργοποιήσεις παραμένουν σχετικά σταθερές, γεγονός που υποδηλώνει ότι το δίκτυο **έχει μάθει πιο αφηρημένες και ανθεκτικές αναπαραστάσεις**, λιγότερο ευαίσθητες σε τοπικές διαταραχές.

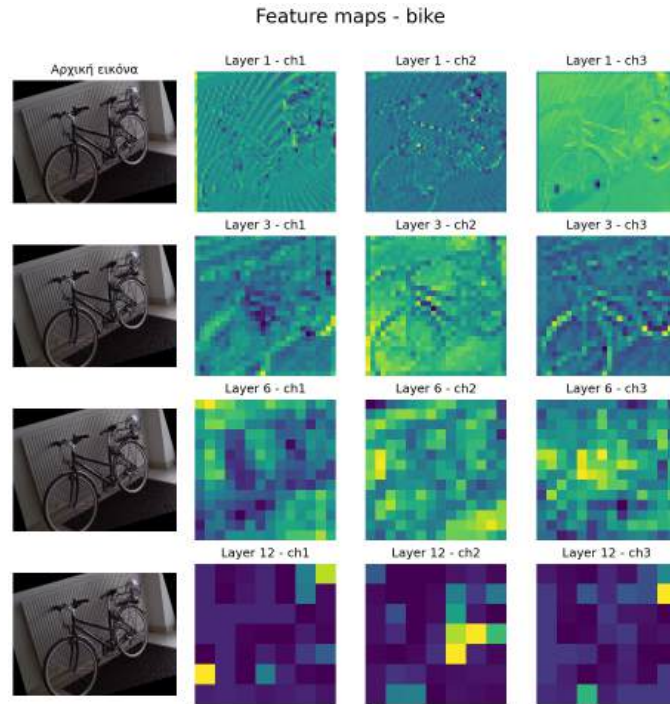
3.2 με Περιστροφή

Αντίστοιχα, με την προσθήκη θορύβου, η απόδοση του MobileNet μειώθηκε στο 94.897%, έναντι του αρχικού 95.310% χωρίς παραμόρφωση, δηλαδή το μοντέλο μας **ανταποκρίθηκε καλύτερα στην περιστροφή από ότι στο θόρυβο**.

Φυσικά, η αλλαγή στην απόδοση είναι και στις δύο περιπτώσεις ελάχιστη, με την περιστροφή να την επηρεάζει λιγότερο. Αυτό πιθανόν οφείλεται στο γεγονός ότι η **περιστροφή δεν αλλοιώνει τη δομή της εικόνας ούτε τα βασικά της χαρακτηριστικά**, όπως οι ακμές, αλλά απλώς αλλάζει τον προσανατολισμό τους. Έτσι, τα νευρωνικά δίκτυα μπορούν να αναγνωρίσουν ευκολότερα τα ίδια μοτίβα, σε αντίθεση με άλλες παραμορφώσεις που μεταβάλλουν πιο έντονα την τοπική πληροφορία.

Στη συνέχεια, εξετάζουμε την επίδραση της περιστροφής στα feature maps του δικτύου. Στα παρακάτω σχήματα παρουσιάζονται οι αντίστοιχες ενεργοποιήσεις για τις ίδιες εικόνες, μετά την εφαρμογή περιστροφής:





Παρατηρούμε ότι, σε γενικές γραμμές, τα feature maps διατηρούν παρόμοια μορφή με την περίπτωση των αρχικών εικόνων. Αυτό υποδηλώνει ότι το δίκτυο είναι σε μεγάλο βαθμό **ανθεκτικό σε γεωμετρικούς μετασχηματισμούς**, όπως η περιστροφή, και είναι ικανό να εξάγει παρόμοια χαρακτηριστικά ανεξάρτητα από τον προσανατολισμό της εικόνας.

Ωστόσο, ενδιαφέρον είναι ότι στο **Channel 3 του Layer 1** (κυρίως στην εικόνα του ανθρώπου), περιοχές που αντιστοιχούν στο μαύρο φόντο γύρω από την περιστραμμένη εικόνα εμφανίζουν υψηλή ενεργοποίηση. Αυτό συμβαίνει επειδή το φίλτρο ανταποκρίνεται έντονα στις απότομες αλλαγές φωτεινότητας, όπως τα όρια ανάμεσα στην εικόνα και στο μαύρο φόντο που δημιουργεί η περιστροφή. Η ομοιομορφία της μαύρης περιοχής μαζί με τα όρια αυτά κάνει ολόκληρη την περιοχή να φαίνεται “κίτρινη” στο feature map, δείχνοντας ότι το φίλτρο εντοπίζει τις περιοχές με έντονη αλλαγή φωτεινότητας.

Η παρατήρηση αυτή είναι ιδιαίτερα σημαντική, καθώς δείχνει ότι τα αρχικά layers δεν ανιχνεύουν μόνο χαρακτηριστικά του αντικειμένου, αλλά και **τεχνητά όρια (artifacts)** που προκύπτουν από την επεξεργασία της εικόνας, όπως τα μαύρα περιθώρια που δημιουργούνται από την περιστροφή. Σε αντίθεση με αυτό, στα βαθύτερα layers δεν παρατηρείται σημαντική διαφορά σε σχέση με την αρχική εικόνα, κάτι που στηρίζει το συμπέρασμα ότι οι βαθύτερες αναπαραστάσεις είναι πιο αφηρημένες και λιγότερο ευαίσθητες σε τέτοιες μεταβολές.

Συγκεντρώνοντας τα αποτελέσματα όλων των πειραμάτων στον παρακάτω πίνακα, προκύπτουν ορισμένα σαφή συμπεράσματα. Το **MobileNet υπερέχει σε όλες τις συνθήκες**, τόσο σε κανονικές εικόνες (95.310%) όσο και υπό παραμορφώσεις. Από τους τοπικούς περιγραφητές, ο συνδυασμός **corners+SURF** είναι ο πιο ανθεκτικός, με τη μικρότερη πτώση και στις δύο παραμορφώσεις. Συμπερασματικά, αν υπάρχει δυνατότητα υψηλότερου υπολογιστικού κόστους, η καλύτερη επιλογή για οποιοδήποτε πρόβλημα κατηγοριοποίησης εικόνων, ανεξάρτητα από τις συνθήκες λήψης, είναι η χρήση του νευρωνικού δικτύου.

Ανιχνευτής	Περιγραφητής	Κανονικό	Θόρυβος	Περисτροφή
Multi-scale corners	SURF	74.207%	72.414% (-1.793%)	68.690% (-5.517%)
Multi-scale corners	HOG	64.000%	56.276% (-7.724%)	52.966% (-11.034%)
Multi-scale blobs	SURF	73.793%	71.448% (-2.345%)	67.586% (-6.207%)
Multi-scale blobs	HOG	62.897%	57.241% (-5.656%)	54.207% (-8.690%)
MobileNet	-	95.310%	93.241% (-2.069%)	94.897% (-0.413%)

Table 4: Σύγκριση απόδοσης όλων των μεθόδων υπό κανονικές συνθήκες, με θόρυβο και με περιστροφή (Οι τιμές σε κόκκινο δείχνουν την πτώση σε σχέση με τις κανονικές συνθήκες)